

Fine-grained Role-Based Access Control in Data Lakes by Encryption

Andreas Severin Hauch Trøstrup
Computer Science
IT University of Copenhagen
Copenhagen, Denmark
atro@itu.dk

Lucas Frey Torres Hanson
Computer Science
IT University of Copenhagen
Copenhagen, Denmark
luha@itu.dk

STADS: KISPECHISE

Abstract—True separation of compute and data in data lakes comes at the cost of coarse access control. Authorization services, like Lakekeeper, introduce access control for any query engine, but limit authorization to the file level. On the other hand, “fully managed all-in-one cloud platforms” like Snowflake offer fine-grained access control only within their own platform – defeating the purpose of separating compute and data.

We propose Object Store Wrapper Service (OSWS), a system that enforces column-level role-based access control at the Object Store layer using Parquet Modular Encryption with envelope encryption, and still supports query engines that are already S3-compatible, with no modifications.

Our evaluation, based on end-to-end latency benchmarks, shows that OSWS adds low absolute overhead for reading small Parquet files. With a warm DEK cache, a 0.5 MB Parquet file reaches a *p95* latency of 37 milliseconds, remaining practically fast, though still adding a $\sim 2.5\times$ overhead compared to direct S3 access. However, for larger files, the current implementation becomes impractically slow. We reflect on how these issues could be addressed through different fundamental design changes in OSWS.

OSWS demonstrates that encryption-based fine-grained access control at the Object Store layer is a feasible approach for data lakes, though query engines that cache Parquet metadata pose limitations; however, those issues are addressable.

I. INTRODUCTION

Today, when you want to enforce access control in a data lake, you have two options. *Option 1*: Select a “fully managed all-in-one cloud platform” that integrates with data lakes and provides fine-grained access control (FGAC). *Option 2*: Use a service which governs access to the data lake, but limit access control to the file level. So either you choose *Option 2* and limit who the data can be shared with, based on whether they should only be able to access part of the data, or choose *Option 1* and limit the query engines that clients can use.

Object Stores, such as Amazon S3, which can be used as data lakes, do not understand a granularity finer than file level. This limitation prevents external access control services from governing access to a finer granularity, such as column-based. If, instead, the mechanism for access control was moved within the data lake itself, by leveraging column-based encryption in Parquet files, this could enable fine-grained access control. In the research project “Reviewing options for fine-grained Role-Based Access Control in Data Lakes”, Trøstrup and Hanson [1], a solution called Object Store Wrapper Service (OSWS) was proposed.

The idea of OSWS is to encrypt the Parquet files stored inside the data lake using Parquet Modular Encryption (PME), which supports encrypting each column. The keys will be stored in a key vault and in their wrapped format within the Parquet files, and will be mapped to roles, to limit which roles have access to specific columns. OSWS will expose an S3-compatible API, which will both handle decrypting and encrypting Parquet files for the clients, whilst ensuring they only have access to reading their authorized columns.

In this thesis, a proof-of-concept (PoC) of OSWS will be created to prove that a system supporting external query engines, such as DuckDB, without modifications, can use OSWS; however, it was quickly realized interoperability with the “fully managed all-in-one cloud platforms” depends on the providers allowing custom S3 endpoints. After developing the PoC and coupling a query engine to it, experiments will be conducted to evaluate the performance and functionality, compared to a plain S3-compatible Object Store. Metrics include end-to-end read and write latency in relation to the size of the Parquet file, introduced by OSWS. The data sizes will be based on real-world datasets.

A. Research Questions

Is it possible to create a wrapper service that ensures access control on column-level using role-based-access-control (RBAC) and has interoperability with existing query engines and “fully managed all-in-one cloud platforms” without modifying them?

B. Objectives

This research paper will have the following objectives:

- 1) Create a PoC of OSWS using an underlying S3-compatible Object Store
- 2) Ensure OSWS enforces access control using RBAC and encryption
- 3) Make it compatible with “fully managed all-in-one cloud platforms”
- 4) Make it compatible with “external query engines”
- 5) Benchmark OSWS to measure the latency it adds.
- 6) Reflect on design decisions and which alternatives would have been better

II. RELATED WORK

A. Reviewing options for fine-grained Role-Based Access Control in Data Lakes

This thesis is a continuation of the research paper titled “Reviewing options for fine-grained Role-Based Access Control in Data Lakes”, Trøstrup and Hanson [1].

This showed that there is a gap within the data lake field regarding access control. One option is to use “fully managed all-in-one cloud platforms”, such as Databricks Unity Catalog or Snowflake, to provide granular control of what clients can see, with the platform’s own built-in query engine. The other option is to use solutions using vended credentials, such as Lakekeeper or Apache Polaris; essentially, providing temporary credentials to clients, who use whatever external query engine they want to use, but in exchange, only file-level granularity can be controlled.

Based on this, OSWS was proposed to be able to provide both columnar access control and let clients use whatever external query engine they wanted to use.

This was the foundational work to reflect on how OSWS could theoretically work.

After beginning to implement OSWS, discoveries revealed that it did not work as initially planned, and as described in the research paper.

The biggest change from the research paper is that OSWS does not support external query engines decrypting locally, but has to get the decrypted Parquet through OSWS.

As also mentioned later, “fully managed all-in-one cloud platforms”, such as Snowflake [2] or Databricks Unity Catalog [3], are not guaranteed to be supported as they would have to whitelist a URL on which OSWS runs, and have thus not been able to be tested. Lakehouse tools like DuckLake are not supported either because they cache metadata, and the Parquet file is modified in OSWS, which will also be discussed later.

Though this is an issue that could be addressed in the future, it will require OSWS to adopt a different system design than the one it currently has.

B. Membrane: A Cryptographic Access Control System for Data Lakes

Membrane, referenced in Trøstrup and Hanson [1], is a proposal by Kumar et. al. [4] to ensure both FGAC at the cell level, and encryption-at-rest in data lakes using a custom format, thus resembling what OSWS attempts to achieve. Besides making the original client responsible for handling the original encryption keys to provide access to others, it also forces the query engine to download the whole table to decrypt it with the provided view on its machine. The advantage of this is that once the query engine has the data, given the view, they only have to make computations on the relevant data. This differs from OSWS, where it actually runs computations on all data, but instead only gives the query engine the finalized data. For Membrane, the overhead of the querying time stems from the size of the view, whereas for OSWS it stems from the size of the Parquet file. Membrane also has finer-granularity access-control than OSWS, cell-

level vs column level, but this comes at the cost of using a proprietary format.

C. One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale

While Membrane offers cell-level granularity at the expense of a custom format, Shang et. al. [5] proposes a system more like OSWS, in that it enforces access control in the storage layer. In their paper, they present a system for fine-grained encryption of Parquet files at Uber’s scale, solving encryption-at-rest and access control as a combined problem, similar to OSWS. Their approach uses column-specific encryption keys stored in KMS, with readers and writers performing cryptographic operations directly through a Parquet library extension.

The key difference is where the cryptographic operations occur. The Uber system requires extending the Parquet reader/writer libraries, which internally use PME, used by the external query engines, making it intrusive to adopt. OSWS performs all cryptographic operations in the Encryption Gateway, exposing a standard S3 API so that unmodified query engines can access data without library changes. The trade-off is that OSWS introduces a network hop and full-file processing overhead, while Uber’s approach avoids both but requires per-engine integration.

III. BACKGROUND

This section will describe the terminology used within this thesis to ensure consistency. A lot of the definitions will be taken directly from the previously written research paper.

A. OSWS

Object Store Wrapper Service (OSWS) is the concept of a layer on top of an Object Store that supports S3-compatible API endpoints. It is meant to provide fine-grained role-based access control (FGRBAC) in data lakes by encryption at the columnar level. Its trust model is based on that it wants to both ensure that clients can only see what they are authorized to, as well as hide the actual data from the storage layer. This will be achieved by using Parquet Modular Encryption (PME), and will only decrypt the columns which are authorized to the client requesting a given Parquet file.

B. Data Lake

A “data lake” is a centralized repository of data, which could be structured, semi-structured or unstructured. [6] [Respectively meaning that the data either has a predefined schema or does not have a fixed schema. [7]] It differs from a traditional database in that a data lake separates compute and storage layers. The data itself is stored in Object Stores like Amazon S3 [8], in file formats like CSV, Parquet, etc. The compute nodes are simply any query engine, such as Apache Spark [9], that connects to the Object Store and uses the data. This could be a data scientist working on their own machine, who connects to a data lake and runs queries; or it could be

a part of a managed all-in-one platform like Snowflake [...]. As the Object Store just contains raw data, both structured and unstructured, this can make it more complicated to query, especially for large data lakes. One approach to querying is “schema on read”, in which the query engine infers a schema at query time based on the files in the data lake. This is in contrast to “schema-on-write” used in traditional relational databases, where the schema is strictly enforced when writing. This doesn’t work well for data lakes due to the nature of storing raw data in Object Stores. Typically, data lakes introduce a *catalogue* on top of the Object Store that manages file schemas, metadata and snapshots, allowing for structured reads and more performant querying through techniques like partitioning, i.e. filtering unnecessary data. This has also been referred to as a *data lakehouse* . [10]

— Trøstrup and Hanson [1]

C. Parquet

Parquet [11] is a columnar file format supported by many data processing systems.

A Parquet file is structured using row groups, which contain columns, along with a footer which contains metadata. The columns are structured as column chunks, which contain several *data pages*. A data page contains a header, which describes information like the size of the page, and following that, the actual values as bytes. The file footer contains metadata that specifies what is stored in the file and where, that is, where a reader should look to find a specific column, or the statistics of a column (e.g. min/max). [...] The objects that hold metadata, like statistics, and structural information, including physical offsets, such as the footer and page headers, are serialized using Apache Thrift[12], a language-agnostic serialization framework, and referred to as “Thrift Structs”. Essentially, the object that contains the information, for example, a `PageHeader` object [...] or a `ColumnMetaDatum` object [...], is serialized into bytes using Thrift, which can then later be restored to the same object. [13]) Then, to read a Parquet file, the reader de-serializes the Thrift structures to get the necessary metadata, and then reads the actual data values using the information from the metadata.

— Trøstrup and Hanson [1]

D. RBAC

Role-based Access Control (RBAC) [14] is an approach to access control based around assigning permissions to roles, and grouping users into these roles. Access control is then managed by controlling what roles are allowed to do what, instead of managing each user individually. The “Core RBAC” model, as defined by Ferraiolo et. al. [15], consists of five basic elements: *users*, *roles*, *permissions*, *operations* and *objects*. A *user* is some actor, typically a

human user, but could also be an autonomous actor, who needs to be authorized for some set of actions. A user is assigned one or more *roles*. A role is simply some named collection of authorizations; for example, a job title. A role is assigned one or more *permissions*, which are what give authorization to perform an *operation* on a secured *object*. [...] In addition, Core RBAC also describes *sessions*. A *session* is a mapping of one user to a subset of the roles they are authorized for; i.e. a user “activates” one or more roles that have the necessary permissions, when they need to perform a specific operation on an object. This allows for finer control of roles that are active in a given context, to prevent unnecessary access levels. Commonly, a hierarchy is introduced to allow for structuring roles which should inherit some base permissions; for example, a Doctor and Nurse role might inherit permissions from some base “Healthcare Worker” role. This is referred to as “Hierarchical RBAC” [15].

— Trøstrup and Hanson [1]

E. OIDC

OpenID Connect (OIDC) is an identity authentication protocol based on the authorization OAuth 2.0 framework. [16] It provides developers with a standardized, simple way to verify the identity of users trying to access web applications. It allows users to authenticate using their existing accounts previously registered with an OpenID Provider, such as Microsoft Entra ID [17] or Pocket ID [18], which the web application can then contact to verify their identity. Thus, it eliminates the need to implement an authentication layer in the web application.

F. KMS/KV

Key Management Service (KMS) and Key Vault (KV) are both services used to manage keys. [19], [20] KMS, both as a term and service, is widely used within data storage, and is what AWS use to refer to their system, whereas Azure uses KV. This paper will use the term KV, as OSWS has been set up against Azure KV, but could have used AWS KMS. KV is a way to store cryptographic keys. When the cryptographic keys are within the KV, they can no longer be retrieved, and thus, if data has to be encrypted/decrypted, it either has to be sent to the KV or envelope encryption can be used, so the KV is responsible for unwrapping the DEK.

G. PME

Parquet Modular Encryption (PME) is a way to allow granular control of how the Parquet file data and metadata should be encrypted. Its trust model is based on ensuring that the client-side written data is not read by the storage layer. [13] Singular columns can be encrypted, and the footer as well. When encryption is used, serialized Thrift structs are encrypted using the given cryptographic key, and the data pages themselves are encrypted as well.

The encryption algorithms used in PME are the AES standard, and support the standard 128, 192, and 256 bit sizes.

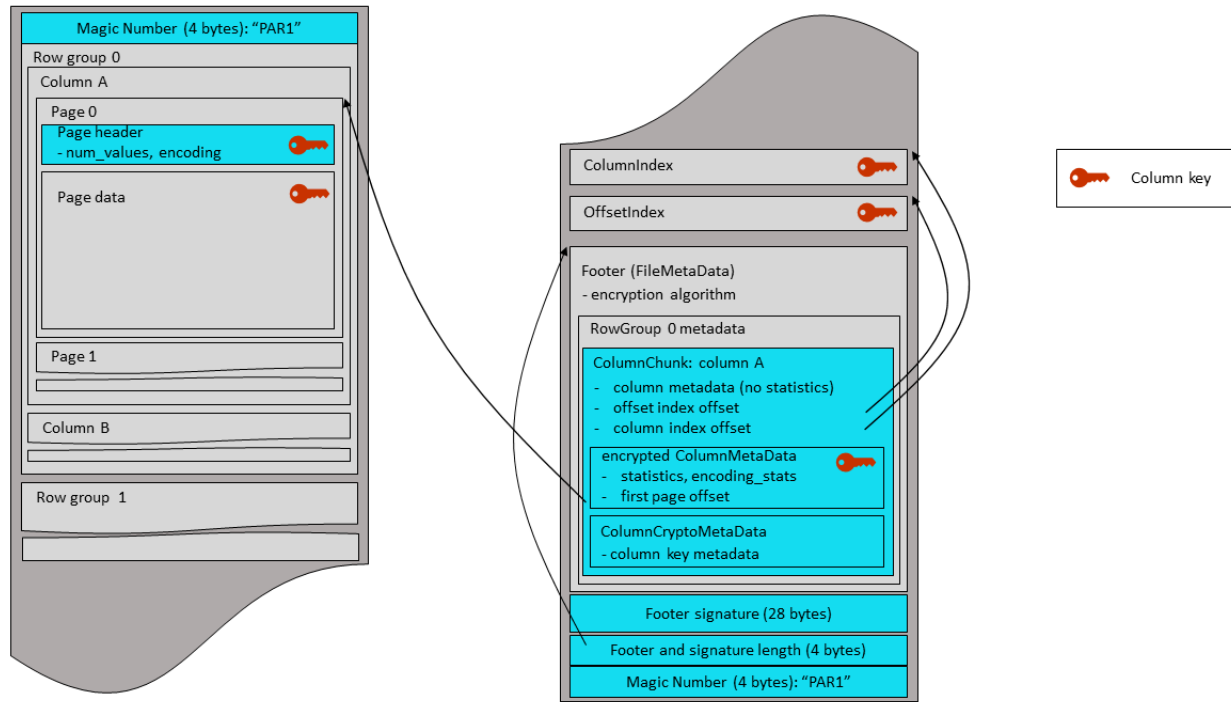


Figure 1. Illustration of a Parquet file using Plaintext footer. The red key denotes what is encrypted with the column key. From [13].

Both AES-GCM [21] and a combination of AES-GCM and AES-CTR [22] can be used, each with its own tradeoffs. AES-GCM provides both data confidentiality and data integrity verification. Because of the integrity verification, the result of encryption is both a ciphertext and an authentication tag. When used for Parquet files, these are stored in their metadata, resulting in a modified file size. This comes at the cost of the speed of encryption and decryption.

On the other hand, AES-CTR only provides confidentiality due to not performing integrity validation, but it is faster, and adds no additional length to the encrypted Parquet file. However, PME does not support a pure AES-CTR mode. Instead, an AES_GCM_CTR_V1 mode is available, where the metadata is encrypted with AES-GCM and the rest with AES-CTR, still resulting in a changed Parquet file size. [13]

In short, AES-GCM provides confidentiality as well as integrity by also signing the ciphertext, and AES-CTR instead focuses on making the encrypted data into a stream cipher, but has no integrity verification due to not signing the ciphertext.

a) Encryption in Parquet:

In PME, column encryption is always enabled, but it provides two options for footer encryption: an encrypted or plaintext footer. Figure 1 shows a Parquet file with plaintext footer enabled. In plaintext footer mode, a ColumnCryptoMetadata struct is added to each column chunk, which contains the necessary metadata to decrypt the column, such as key ID and algorithm. This is shown on the right side of Figure 1. Notably, each column's statistics are moved to an encrypted ColumnMetadata struct, also shown on the right side of Figure 1. This ensures that no information about the values can be gained without decrypting

the columns. Non-sensitive information, like the offsets and other metadata, is stored as-is without encryption.

H. Envelope Encryption (KEK, DEK)

KV does not support retrieving keys, and as a result, OSWS uses envelope encryption. This means that during encryption of a column, cryptographic keys for each are generated, called a data-encryption-key (DEK), which is used to encrypt the columns. Then a single key-encryption-key (KEK) is generated for the whole Parquet file, resulting in less data compared to if each DEK had its own separate KEK within KV. This KEK then encrypts all the DEKs, also called wrapped DEKs, and gets stored in KV. The wrapped DEKs are then stored in the metadata for the Parquet file together with a KEK ID. When the columns are to be decrypted, the wrapped DEKs are sent to KV to be unwrapped, and the columns can be decrypted. [23]

I. TTL

Time-to-live (TTL) is a way to define the lifetime of some item within a dataset. When either the item's TTL is reached or the dataset is full, the item will be evicted. [24] In OSWS, TTL defines the lifetime of unwrapped DEKs in the cache. When the TTL is reached, the data will be removed and has to be re-unwrapped from the KV when needed again. In a NIST paper [25], it is recommended to use two different TTLs, one for admin and one for the rest. This was not fully implemented, and currently defaults to 15 minutes if no configuration changes are made.

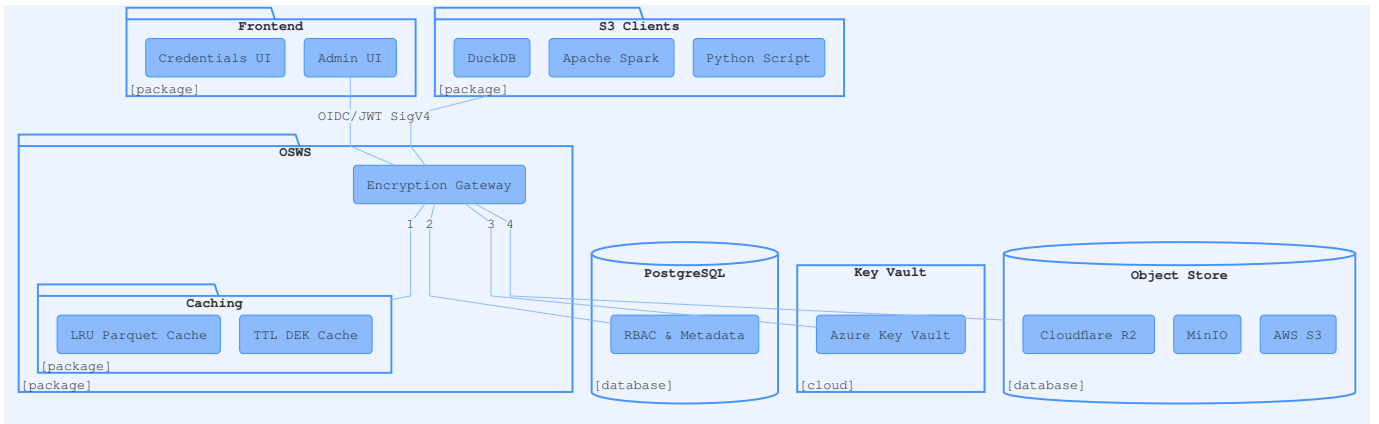


Figure 2. Architecture of OSWS showing components, external services, and communication paths

1 referencing looking up cached DEKs and Parquet files | 2 referencing RBAC calls | 3 referencing DEK wrap or unwrap | 4 referencing S3 API calls

J. Fully managed all-in-one cloud platforms

In this paper, “fully managed all-in-one cloud platforms” will be used to define solutions such as Snowflake and Databricks Unity Catalog. Access control within those solutions is managed by the query engines together with the catalogue. This means the query engine is trusted to filter out rows that are not permitted, and it then masks or removes those values, which is possible as the platforms themselves manage it.

K. External Query Engines

“External query engines” will be used to refer to query engines that operate separately from the data lake. This is one of the key features of data lakes; the compute and data layers can be separated. Compared to a “fully managed all-in-one cloud platform”, the query engine is not managed by a platform and can be any with the ability to fetch Parquet files from an S3-compatible API and perform operations on these. This means the query engine cannot be trusted to filter out unauthorized data, as with “fully managed all-in-one cloud platforms”.

Examples of these tools include Apache Spark [9], and DuckDB [26]. Lakehouse solutions such as DuckLake [27] also exist. However, DuckLake is not just a basic query engine. Besides bringing a query engine, it also provides a catalogue and other features, such as time-travelling. The catalogue is the main difference, providing a way to store metadata about the files, including their size, location, etc, and makes use of “schema on write”; it stores the schema of the file when it writes it.

DuckLake will be included in the definition of “external query engine” as the use in the context of OSWS is the same. The important distinction between a query engine like DuckDB and a DuckLake-like system is that DuckDB infers the schema when it reads, while DuckLake stores it on write, and thus does not need to infer it later. This has a major impact on how they interact with OSWS and will be addressed later.

L. Amazon Signature V4

When clients communicate with Amazon S3 or S3-compatible API’s like Cloudflare R2, an authentication scheme known as AWS Signature V4 [28] is used. A pair of creden-

tials known as the *client ID* and *client secret* is generated by the service, and both parties, client and service, store the information. When requests are made, the request is signed using the *client secret* and the *client ID* is appended as well. The service can find the credentials through the ID and re-signs the request. The request is authenticated if the signatures match.

M. JIT Provisioning

Just-In-Time (JIT) Provisioning is an identity management process handling creating user accounts when they are needed. [29] This is needed during the initial user creation.

IV. SYSTEM DESIGN

In this section, OSWS is designed and the reasoning for the choices made.

A. Architecture Overview

OSWS is designed as a wrapper for S3 and thus provides S3-compatible API endpoints for clients. The purpose of OSWS is to provide column-level access controls to S3, whilst not modifying the S3 API. This ensures that most S3-compatible query engines can use OSWS without modification and will only read what they are supposed to.

The system is built on ASP.NET Core 10 using the minimal API pattern [30], which replaces traditional MVC with lightweight endpoint definitions. OSWS uses: PME via. Parquet Sharp, PostgreSQL for RBAC metadata, Azure KV for key management, and a configurable S3-compatible Object Store (tested with Cloudflare R2 and Digital Ocean Spaces for benchmarking).

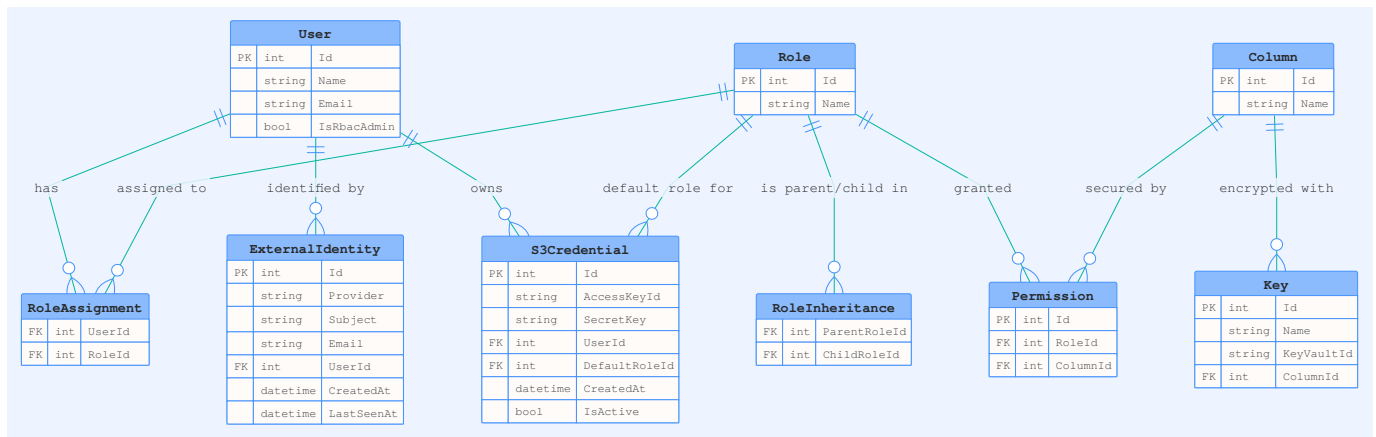


Figure 3. Entity Relationship Diagram showing the design of the database

TABLE I
OSWS SYSTEM COMPONENTS AND THEIR RESPONSIBILITIES

Component	Responsibility
Encryption Gateway	S3-compatible API host. Handles incoming S3 requests, enforces AWS Signature V4 authentication, and orchestrates encryption, decryption, and column filtering.
Parquet Solver	Encrypts and decrypts Parquet files using PME with Parquet Sharp. Generates DEKs, wraps them via the Key Manager, and masks unauthorized columns.
Key Manager	Manages cryptographic keys through Azure KV (or an internal provider). Wraps/unwraps DEKs using RSA-2048 KEKs stored in the vault.
RBAC Store	PostgreSQL database storing users, roles, role assignments, role inheritance, column permissions, and S3 credentials. Evaluated per-request to determine authorized columns.
Frontend	React web UI for managing S3 credentials, and, for admins, managing roles, permissions, and users. Uses OIDC authentication and provides a SQL-like query editor for RBAC operations.

OSWS consists of five main components, each with its own responsibility for handling incoming requests, handling outgoing requests, or storing/modifying data. A summary of each can be seen in Table I. In addition, Figure 2 shows external services as well, and how they interact with the core OSWS system. The **Frontend** is for users to interact with the API. For regular users, it can be used to create S3 credentials for use with the **encryption gateway**. For admins, it allows RBAC management, handling roles, permissions, and users within the OSWS system, specifically within the PostgreSQL **RBAC store**, the database that stores these relations. Here, it can be defined what columns a user can access later on and, by extension, which columns will be masked.

The **encryption gateway** is the entry point for retrieving and putting files into the actual Object Store. It is responsible for delegating work to the other components. Whenever it receives a GET request, it will first get the file from the Object Store and see which columns the client is able to read by using the RBAC store. Users authenticate using credentials

generated in the frontend. Then it will use the **Parquet Solver**, which in turn will handle trying to identify which DEKs are in the Parquet file. It can then go through the **Key Manager**, to get the unwrapped DEKs based on permission level; it will check the DEK cache first, and if not stored, go to KV to get them unwrapped there, and store them for later use. After the key manager returns the unwrapped DEKs, the Parquet Solver can decrypt the columns and mask out the columns that the client is not authorized to view. Now the encryption gateway can return the Parquet file, showing only the information that the client has access to.

When the encryption gateway gets a PUT request, it first gets the user's active role. For the PoC, this was simply chosen to be the user's first available role, but optimally should have been selectable by the client. Then the Parquet solver is given the file and role, so it can generate new DEKs per column and request a KEK for the Parquet file, through KV. The RBAC store is updated with the new columns and key references, and the file is encrypted afterwards. The DEKs are wrapped by the KEK using KV, to then save the wrapped version inside the Parquet metadata.

This allows OSWS to ensure fine-grained role-based access control in data lakes, but of course, there are more in-depth design choices that have been made to make this work, both for better and worse.

B. Design Choices

a) Database Design:

An Entity Relationship Diagram describing the database design can be seen in Figure 3. The database is mostly used for RBAC metadata; however, it also stores credentials used for AWS Signature V4 request signing, and external identity information from the OpenID Provider(s).

b) RBAC Metadata:

In Section III.D, RBAC is defined to consist of: *users*, *roles*, *permissions*, *operations*, *objects* and *sessions*. OSWS implements RBAC mostly following this, but it differs in a few ways. In Figure 3, the core RBAC entities are shown: the tables User, Role, RoleAssignment, RoleInheritance, Column, and Permission. User and Role have a many-to-many relationship through RoleAssignment. Likewise, Role and Column have

a many-to-many relationship through *Permission*. The *Column* table can be seen as the *object* of Core RBAC, as it is currently the only secured object to which access is controlled. There is, however, no concept of *operations*, only whether or not access is granted. There is also no formal concept of *sessions* as described by Ferraiolo et. al. [14]; however, when a user is authenticated through their S3 Credential, their assigned roles are loaded. In this way, it can be seen as an activation of all the users assigned roles.

c) *Role Hierarchy*:

```
WITH RECURSIVE effective AS (
  SELECT ra."RoleId" AS "Id"
  FROM "RoleAssignments" ra
  WHERE ra."UserId" = {userId}
  UNION
  SELECT ri."ChildRoleId"
  FROM "RoleInheritances" ri
  JOIN effective e
  ON ri."ParentRoleId" = e."Id"
)
SELECT DISTINCT r."Id", r."Name"
FROM "Roles" r
JOIN effective e ON r."Id" = e."Id"
```

Listing 1. A recursive SQL query to get the full set of effective roles given a *userId*.

OSWS also supports Hierarchical RBAC through the *RoleInheritance* table. This is a self-referential table in which a hierarchical relation is created through the *ParentRoleId* and *ChildRoleId* foreign keys, so that a *parent* inherits from a *child*, see *RoleInheritance* in Figure 3. This means that all permissions of *child* also become permissions of *parent*. To retrieve the full set of effective roles for a user, a recursive SQL query traverses the hierarchy. This can be seen in Listing 1.

d) *Column-Level Permissions*:

Each *Permission* record maps a *Role* to a *Column*, giving fine-grained control over which columns a user can decrypt.

Example: Given a Parquet file *P1* with columns *name*, *age*, and *social_security_number*:

- Role *R1* grants access to: *name*
- Role *R2* grants access to: *age*, *social_security_number*

A user *U1* assigned *R1* sees valid *name* values; *age* and *social_security_number* appear as nulls. A user *U2* assigned *R2* sees the inverse: valid *age* and *social_security_number*, with *name* as null.

Furthermore, using the Role Hierarchy, role *R2* can be made to inherit the permissions of *R1*. Thus, user *U2* would see all columns, without needing the *R1* role directly.

e) *Encrypted File Cache*:

Given the configuration of OSWS, the Parquet files fetched from the Object Store can be cached. It uses the Least Recently Used (LRU) policy to cache the encrypted Parquet files on the local file system. Each Parquet file is then keyed with a tuple of the bucket name and its key, meaning that it uses the bucket name and the full path and file name to save the Parquet file, reflecting how S3 handles its internal naming.

f) *DEK Cache*:

Unwrapped DEKs are cached in-memory to avoid repeated KV calls, as it is one of the major latency attributes, discussed later. The cached DEK gets a unique identifier, which, along with the KEK identifier, is used as the cache key. The cache enforces a maximum capacity and evicts the expired entries first, and then the oldest entries by their TTL.

g) *Envelope Encryption*:

To encrypt the Parquet columns, OSWS needs to use DEKs and KEKs to achieve envelope encryption [31]. This deviates from the original proposed solution in Trøstrup and Hanson [1], as KV does not support key retrieval, and the overhead of sending a Parquet column to KV each time for decryption and encryption is high. By using envelope encryption, OSWS can cache the unwrapped DEKs with a specified TTL to allow quicker decryption. This results in a solution with lesser overhead.

h) *Parquet Solver*:

Parquet Solver handles the cryptographic operations for the Parquet files. Important design decisions made here are that it uses Parquet Sharp, specifically version 21.0.0. [32] Parquet Sharp was chosen as it supports PME, but does not support partial decryption/reads of Parquet files and thus has to decrypt and copy over all columns into a new Parquet file. For the unauthorized columns, null, empty strings, or zeros (dummy data) have to be copied instead.

The choice of building around PME and Parquet Sharp turned out to be one of the design choices that later created more problems than it solved. This will be discussed in more detail later.

i) *Authentication*:

Since OSWS provides an S3-compatible API, authentication must follow the Amazon Signature V4 scheme as well. And, since OSWS rewrites the content and re-routes the request, the original signature is no longer valid, so OSWS needs its own credentials as well. Thus, OSWS both needs to verify the client's request, and correctly sign the new request to the Object Store.

To support AWS Signature V4, a custom SigV4 authentication handler was implemented. Using this, the corresponding *S3Credential* record is then looked up in the internal PostgreSQL database, displayed in Figure 3. The signature is verified using the stored secret key. The signing process follows the specification defined in AWS Signature V4 [28]. On success, the request is authenticated, and the user's information is resolved.

j) *OIDC*:

The React frontend, for handling RBAC, uses OIDC for the authentication flow. The backend has been made to be OIDC provider-agnostic, and the users can bring their own provider of choice. The desired provider can be changed through the application's configuration files. While building the prototype, Pocket ID¹ was used as an OIDC Provider, due to being open-source and simple to set up.

On the first login, the *api/me* endpoint triggers JIT provisioning, and a new *User* record and *ExternalIdentity*

¹<https://github.com/pocket-id/pocket-id>

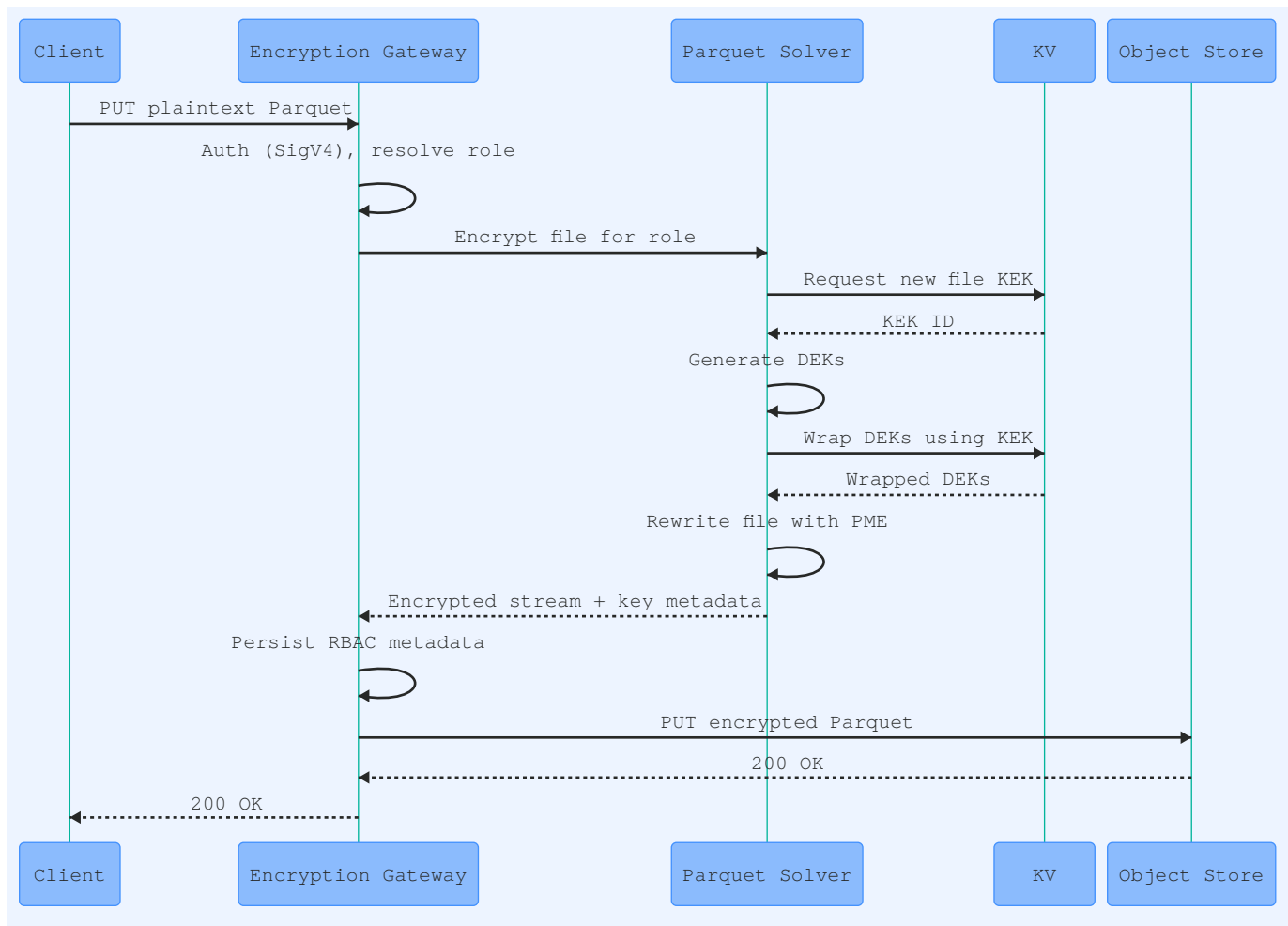


Figure 4. Encryption flow of OSWS

record are created in the database. Subsequent logins will synchronize the OIDC provider's claims. The admins of the system can then define what the given clients can access in the admin frontend.

k) Key Hierarchy:

The KEK sizes are specified within the KV that the admin chooses. Azure KV is used to store the KEKs, chosen due to pricing, but is able to be switched out. RSA-2048 keys are used with the RSA-OAEP-256 [33] encryption scheme for wrapping and unwrapping. These are generated in the KV itself.

KV will then be called by OSWS to wrap and unwrap the DEKs, which themselves are stored within the Parquet metadata. The DEKs are created within OSWS as well, and use AES Based on the setup, they can have sizes 128, 192, or 256 bits, and are specifically created during the encryption flow.

l) Frontend Architecture:

A frontend for managing credentials and for admins, managing RBAC, was created using TypeScript with React. A user can log in using Pocket ID as an OIDC provider. Here, the user can create credentials to be used for the S3-Compatible API endpoints. If the user is an RBAC Admin, they get access to the admin panel for managing roles. The admin panel

includes a table view for viewing currently existing users, roles, columns and permissions.

```

CREATE ROLE admin;
=> POST /api/admin/roles { name:
"admin" }
CREATE ROLE intern;
=> POST /api/admin/roles { name:
"intern" }
GRANT intern TO ROLE admin;
=> POST /api/admin/roles/1/inherit/2
GRANT ACCESS ON name TO intern;
=> POST /api/admin/columns/1/roles/1
GRANT ACCESS ON ssn TO admin;
=> POST /api/admin/columns/2/roles/2
GRANT admin TO USER alice;
=> POST /api/admin/users/1/roles/1
  
```

Listing 2. Example statements and their parsed API calls

To interact with the administrative API, a “query editor” was built to manage RBAC operations in an SQL-like syntax, reminiscent of the syntax used to manage permissions in e.g. PostgreSQL.² Using the JavaScript library `peggyjs` [34],

²<https://www.postgresql.org/docs/current/sql-grant.html>

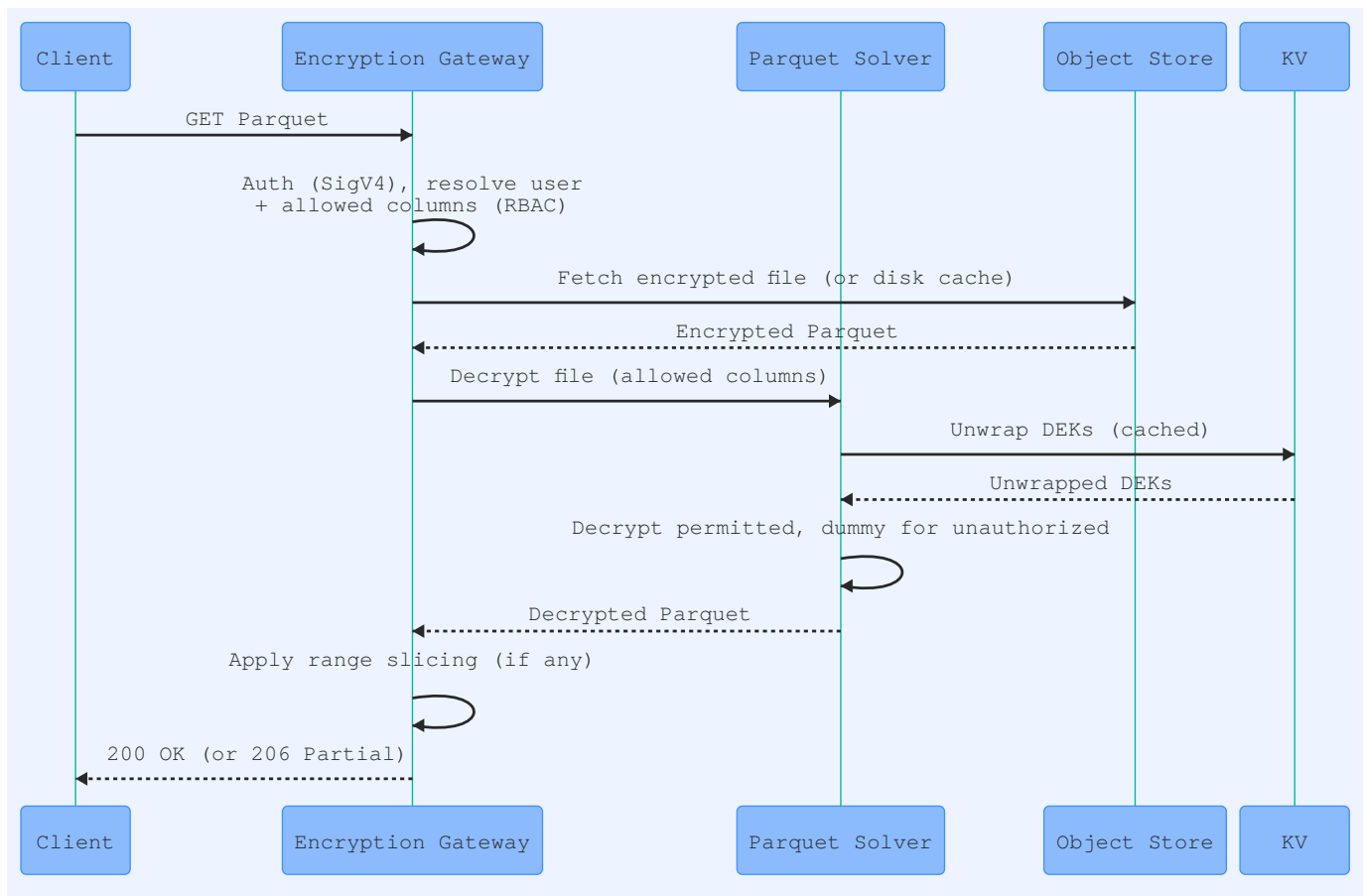


Figure 5. Decryption flow of OSWS

a simple grammar was written to parse statements into API calls. Some example statements to configure RBAC permissions and the API calls they parse can be seen in Listing 2. The API endpoints will be described in more detail in Section IV.D.

C. Walkthrough Examples

a) Encryption Flow:

Figure 4 shows the flow of how encryption works in OSWS, from the client's PUT call to the client receiving an acknowledgement that it has been completed. It follows the given flow:

- 1) Client uploads an unencrypted Parquet file to OSWS – through the encryption gateway
- 2) OSWS checks the client credentials and resolves their first role – which is a simplification and should use a default role
- 3) Encryption Gateway then hands over the work to the Parquet solver
- 4) Parquet Solver uses KV to generate a KEK and gets a reference ID
- 5) Parquet Solver generates a DEK per column
- 6) All the DEKs are sent to KV to get wrapped
- 7) The Parquet file columns are encrypted and copied over to the new Parquet file, and the KEK ID and wrapped DEKs are saved within the Parquet metadata

- 8) Parquet solver hands the work back to the Encryption Gateway with relevant metadata
- 9) Encryption Gateway then persists the metadata in the PostgreSQL database
- 10) OSWS then inserts the Parquet file into the Object Store
- 11) OSWS sends back the acknowledgement that it has been inserted

b) Decryption Flow:

Figure 5 shows the flow of how decryption works in OSWS, from the client's GET call to them receiving a Parquet file back. It follows the given flow:

- 1) Client requests a Parquet file to OSWS, through the encryption gateway
- 2) OSWS checks the client credentials and resolves which roles they have access to
- 3) OSWS then fetches the full Parquet file from S3 – or from the disk cache if available

- 4) OSWS then starts to work on decrypting the file, given the allowed columns
 - a) For each authorized column, it goes to KV to get the related DEK unwrapped - if not already present in the in-memory DEK cache
 - b) The columns are now decrypted and copied into a new Parquet file, and for the ones where the unwrapped DEK is not there, a dummy column will be inserted in its place
- 5) The Parquet file is now created, and the whole file can now be returned to the client, or sliced in memory if they only requested a range

This also highlights that OSWS does not use range requests itself when querying S3, but provides the clients the option to use range requests, meaning that regardless of which range the client queries, OSWS fetches the whole Parquet file and then slices it for them.

D. API

OSWS exposes three different endpoint groups. These are either related to:

- Being S3 compliant and providing the endpoints to interact with the data lake
- Managing credentials for authentication
- Managing RBAC and access control

TABLE II
S3-COMPATIBLE ENDPOINTS PROVIDED BY OSWS

Method	Path	Description
GET	/	List buckets
GET	/ {bucket}	List objects in a bucket
PUT	/ {bucket}	Create bucket
DELETE	/ {bucket}	Delete bucket
POST	/ {bucket} ?delete	Delete multiple objects in bucket
HEAD	/ {bucket}	Retrieve bucket metadata
HEAD	/ {bucket} / {key}	Retrieve object metadata
GET	/ bucket} / { *key}	Retrieve an object (with decryption and column filtering for Parquet)
PUT	/ {bucket} / { *key}	Store an object (with encryption for Parquet)
DELETE	/ {bucket} / { *key}	Delete an object

a) S3-Compatible Endpoints:

For the S3 endpoints, only a subset of endpoints has been implemented to ensure Object Store operations would work and be S3 compatible. The endpoints can be seen in Table II. This enables listing what is in the Object Store, and also the create, update, and delete operations.

Currently, non-Parquet files pass through the encryption step, and the endpoints defined are only to allow operations made by most query engines, and thus suffice for making a PoC.

b) Application Endpoints:

Usually, when using S3, the AWS IAM solution is used. But due to the previously mentioned rerouting limitation and

OSWS signage, in Section IV.B.i, it has to provide endpoints for the clients to use to authenticate against. The endpoints provided can be seen in Table III, and include listing the existing credentials, creating new ones, and deleting them.

TABLE III
APPLICATION ENDPOINTS PROVIDED BY OSWS FOR WEB FRONTEND

Method	Path	Description
GET	/api/me	Current user profile (triggers JIT provisioning)
GET	/api/credentials	List the user's S3 credentials
POST	/api/credentials	Create a new S3 credential (access key + secret key)
DELETE	/api/credentials/ {id}	Revoke a credential (soft delete)

TABLE IV
ADMIN ROLE MANAGEMENT ENDPOINTS PROVIDED BY OSWS

Method	Path	Description
GET	/api/admin/columns	Get all columns
POST	/api/admin/columns/ {columnId} / roles/ {roleId}	Grant access on column with ID columnId to role with id roleId
DELETE	/api/admin/columns/ {columnId} / roles/ {roleId}	Revoke access on column with ID columnId to role with ID roleId
GET	/api/admin/roles	Get all roles
POST	/api/admin/roles	Create a role
DELETE	/api/admin/roles/ {id}	Delete role with ID id
POST	/api/admin/roles/ {parentId} / inherit/ {childId}	Make role with ID parentId inherit role with ID childId
DELETE	/api/admin/roles/ {parentId} / inherit/ {childId}	Delete inheritance of role parentId from role childId
GET	/api/admin/users	Get all users
POST	/api/admin/users/ {userId} / roles/ {roleId}	Grant role with ID roleId to user with ID userId
DELETE	/api/admin/users/ {userId} / roles/ {roleId}	Revoke role with ID roleId from user with ID userId

c) Administrative Endpoints:

Endpoints for admin functionality can be seen in Table IV. These are restricted to only users who have the IsRbacAdmin flag, seen in the User table in Figure 3. This field is populated from the ClaimsPrincipal when provisioning the user in the /api/me route. Thus, it requires the OIDC Provider to include this claim for users who should have admin access. Alternatively, it can be set manually.

E. Limitations

OSWS set out to provide columnar access control to external query engines and “fully managed all-in-one cloud

platforms” by encrypting columns individually using PME, provided through the Parquet Sharp library. But due to early design decisions, this was not possible. As OSWS uses Parquet Sharp and stores cryptographic keys within the metadata of the Parquet files, a few issues appear with some query engines. First, Parquet Sharp uses Apache Arrow, which itself supports writing metadata. [35] This in itself makes sense, as it is possible by looking at e.g. `created_by` to see who created the Parquet file. [36] But as Parquet Sharp needs to write when encrypting the Parquet file, the file is essentially changed with new metadata, and it will no longer be the same Parquet file that the external query engine inserted. This becomes a problem for query engines that store metadata themselves when writing. Since OSWS rewrites the file, the query engine will have outdated information, which can break range requests.

On top of that, OSWS saves cryptographic-related metadata within the column metadata, and that is yet another modification. As such, for this to work with all query engines, it would have to be ensured that all ranges of data remain the same, also after encryption.

The same issue is likely prevalent on most “fully managed all-in-one cloud platforms”, but it is not possible to test, due to it needing to have a URL whitelisted by the providers to test the OSWS system. But given documentation of e.g. Snowflake, it seems to point towards incompatibility. [37], [38]

In Section VII, this will be discussed in more depth, along with possible solutions to bridge these two design incompatibilities.

Lastly, initially, it was intended to make OSWS return unauthorized columns in their encrypted form, so that they would not be readable. But due to the limitations in Parquet Sharp, it was not possible, and the current solution is instead to create dummy data entries for the unauthorized columns.

V. METHODOLOGY

To ensure OSWS could live up to the intended use-cases and to check the viability of it, both benchmarking and e2e (end-to-end) tests are needed. This section will cover both the setup of the benchmark and the setup of the e2e test.

All source code is available at github.com/lucasfth/osws, and also describes how to get it running. The version used for testing and benchmarking was V2026.0.0 Alpha³

A. E2E Tests

As one of the goals of the OSWS was to enable third-party query engines to connect without any modification, a test confirming this behaviour is needed.

An e2e test will be created that asserts different query engines can connect and perform queries as normal, with OSWS providing column-level security based on the test user’s roles. The e2e test suite will contain a seeding step that sets up the necessary users and roles. Python scripts will be used to orchestrate setting up roles and permissions, and asserting that the fetched data does not contain unauthorized data. The

test will use the “Titanic” sample dataset⁴ and permissions will be configured so three different roles have limited access, but to different columns. One role will inherit from all roles and have direct access to all other columns, thus having full access. This will then test the role hierarchy feature. Then, the following will be used to fetch the Parquet file through different users and assert that only the permitted columns are viewable:

Python script A Python script using an S3 client like `boto3`⁵ and a Parquet reader like `pyarrow`⁶ to fetch a file from OSWS.

DuckDB A DuckDB instance will query the parquet file through OSWS.

Apache Spark An Apache Spark instance (through PySpark) will query the parquet file through OSWS.

Each of the different tools should assert that only the permitted data is viewable.

B. Benchmarking

All benchmarking of OSWS was run on a Digital Ocean VM with access to 8 cores and 16GB of DRAM, see Appendix 1.a for full specification overview, together with Digital Ocean’s own Object Store (Spaces), which is S3-compatible. For benchmarks using a key vault, Azure KV was used. All services were kept in the same region to further ensure minimal network latency. The test Parquet files used in the following benchmarks can be defined as follows:

- *Parquet Size Tiny*:~500KB Parquet file with 50 columns and 1,000 rows.
- *Parquet Size Small*:~5MB Parquet file with 50 columns and 10,000 rows.
- *Parquet Size Medium*:~125MB Parquet file with 50 columns and 250,000 rows.
- *Parquet Size Large*:~500MB Parquet file with 50 columns and 1,000,000 rows.
- *Parquet Size X-Large*:~1GB Parquet file with 50 columns and 2,000,000 rows.

The five Parquet files are auto-generated by a script that is also available in the source code. The configuration of the files was chosen to reflect a somewhat realistic distribution of different sizes of Parquet files. Various openly available datasets were compared to settle on the choice of 50 columns. For example, several real-world example datasets used for benchmarking, available from the cloud analytics platform ClickHouse, contain fewer than 50 columns. [39] Another is a dataset collecting 1.1 billion taxi and Uber rides in New York City, which contains 51 columns. [40] 50 columns seemed like a realistic number of columns for a large OLAP dataset and was thus used for the sizes.

a) Micro Benchmarks:

The micro benchmarks that will be run will show how each major part of OSWS performs to get insights into how much they add to latency. The following micro-benchmarks will be run:

⁴<https://www.agentsfordata.com/app?sample-id=titanic&tab=local>

⁵<https://github.com/boto/boto3>

⁶<https://github.com/apache/arrow>

³<https://github.com/lucasfth/osws/releases/tag/V2026.0.0-alpha>

TABLE V
MAIN CONFIGURATIONS OF BENCHMARKS THAT ARE TO BE RUN

Category	Benchmark	Configuration	Measurements
E2E	S3 (Spaces)	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS +encryption +caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS +encryption -file caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS +encryption -DEK caching	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
E2E	OSWS -encryption	No. rows: 1,000, 10,000, 250,000, 1,000,000 (50 columns)	Latency
Micro	Permission Service	No. flat hierarchy roles: 4, 64, 256	Latency
Micro	Permission Hierarchy	No. hierarchical depth roles: 0, 4, 16, 64	Latency
Micro	Unwrapping	Key size: 128, 192, 256 bits	Latency
Micro	Decryption	No. rows: 1,000, 10,000, 250,000, 1,000,000, 2,000,000 (50 columns)	Latency

- 1) Permission Service
- 2) Permission Hierarchy
- 3) Unwrapping
- 4) Decryption

Permission Service will measure the latency of how long it takes to get a user's effective roles to the given columns, when there are no inherited roles, i.e. the role hierarchy is flat. The columns stay at a constant 50, the roles assigned to the user are varied from 4, 64, and 256 roles, and they are each assigned, on average, access to 80% of each of the columns.

Permission Hierarchy will measure the latency of how long it takes to get a user's effective roles, now with the role hierarchy introduced. The number of roles assigned is constantly at one, while the number of inherited roles assigned varies from 0, 4, 16, and 64 and again, on average, each role is assigned access to 80% of the columns.

Unwrapping will measure the e2e latency for reading a tiny Parquet file (the small Parquet files are also run but ignored, but can be seen in Appendix 1.b) with a cold cache. The varying key sizes of 128, 192, and 256 bits will primarily affect the symmetric AES-GCM decryption.

Decryption will measure the latency for reading and decrypting a Parquet file, and the keys for decrypting will have been warmed and thus stored in cache; thus, KV is not taken into account here. The Parquet sizes that will be used are the same as with e2e benchmarking, being the tiny, small, medium, large, and extra large.

All of the micro-benchmarks were run 15 times to obtain statistically meaningful results.

b) E2E Benchmarks:

The micro benchmarks show the individual parts' contribution to latency, but the overall performance of OSWS is more important to identify if, from a performance perspective, OSWS is usable within production environments.

As OSWS inherently implements different design choices compared to plain S3, no standardized benchmarking has been chosen, such as Warp⁷, which is otherwise used by multiple other Fortune 500 companies. [41], [42] The reasoning for this is that OSWS implements its own caching layer and uses a KV; as such, the benchmark would have to ensure that it

tests both the throughput of N instances of OSWS running and that the configuration of caching and encryption is considered thoroughly.

OSWS also encrypts each column, resulting in the sizes of the Parquet files having a lot of significance for the end throughput, and Warp does not vary its Parquet file size. This results in inherently different throughputs and numbers, which are hard to compare OSWS against anything else.

Lastly, OSWS, as mentioned, in Section IV.C.b, does not use proper range requests when it queries the Object Store, instead always fetching the full file and slicing after. As a result, it will obviously perform worse for ranged requests compared to direct S3 ones.⁸

Python scripts will then run Parquet PUTs and GETs against the following setups:

- Directly against an S3-compatible store, in this case Digital Ocean Spaces
- Against OSWS with encryption and with caching
- Against OSWS with encryption and no file cache
- Against OSWS with encryption and no DEK cache
- Against OSWS with no encryption and no caching

The metric for this will be end-to-end latency, and for each file, the benchmark will be run for a total of 10 times, both cold and warm, allowing comparison of how slow or speed differences occur with and without caches. As soon as a Parquet file has been fetched once and a DEK has been unwrapped once, they will be warm, and thus in cache.

See Table V for an overview of all benchmarks and their configurations, which will be run.

VI. RESULTS

This section will cover the results of running both the benchmarks and e2e tests described in Section V, and describe their significance.

A. E2E Tests

An e2e test suite was written to test Python using PyArrow, DuckDB and PySpark against OSWS as an S3 endpoint. No modifications were made to the tools outside of changing the

⁷<https://github.com/minio/warp>

⁸A solution to this is proposed within Section VII.

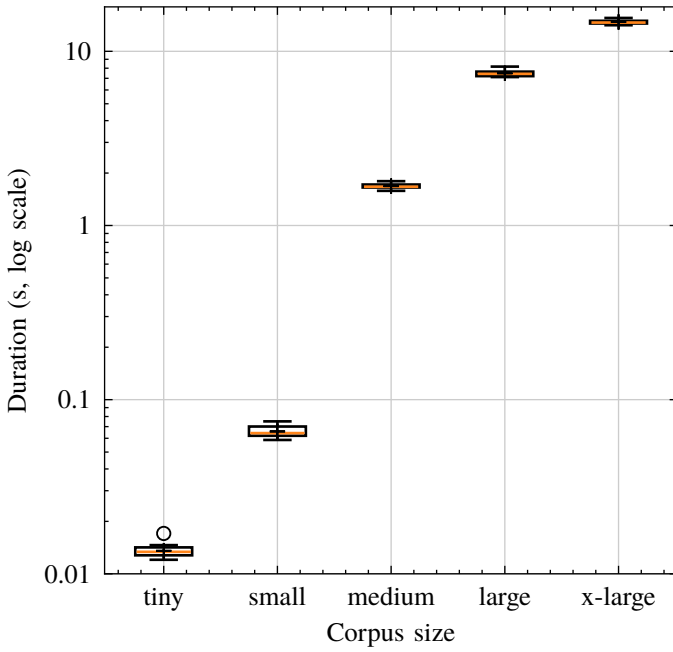


Fig. 6a. Decryption boxplot distribution per parameter (logarithmic y axis)

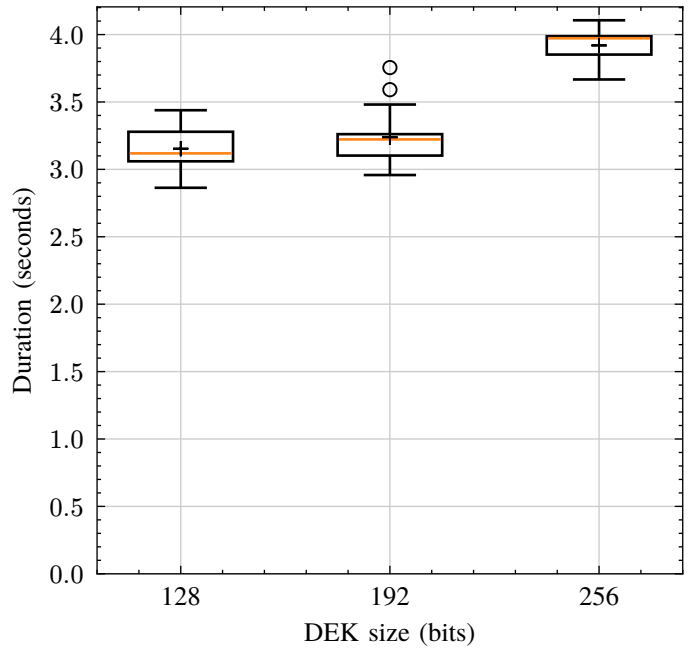


Fig. 6b. KeyUnwrap box-plot distribution per parameter set (tiny corpus only; linear y axis)

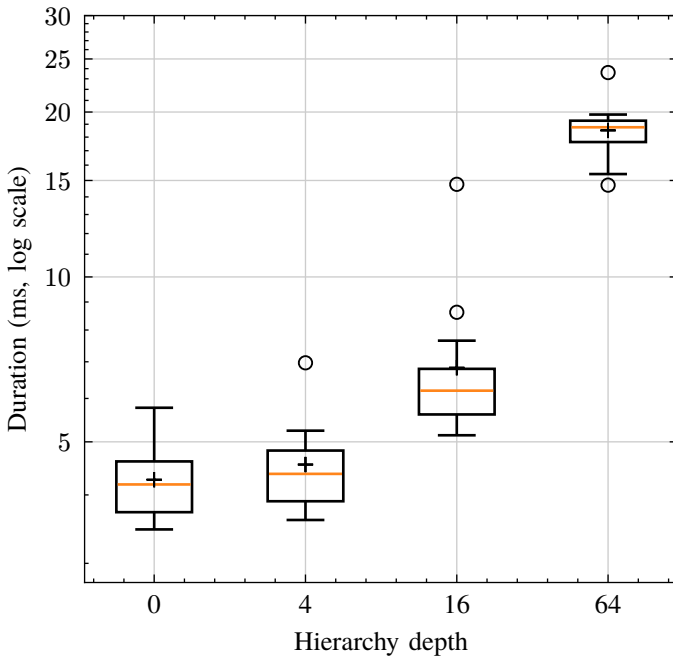


Fig. 6c. PermissionHierarchy box-plot distribution per parameter set (logarithmic y axis)

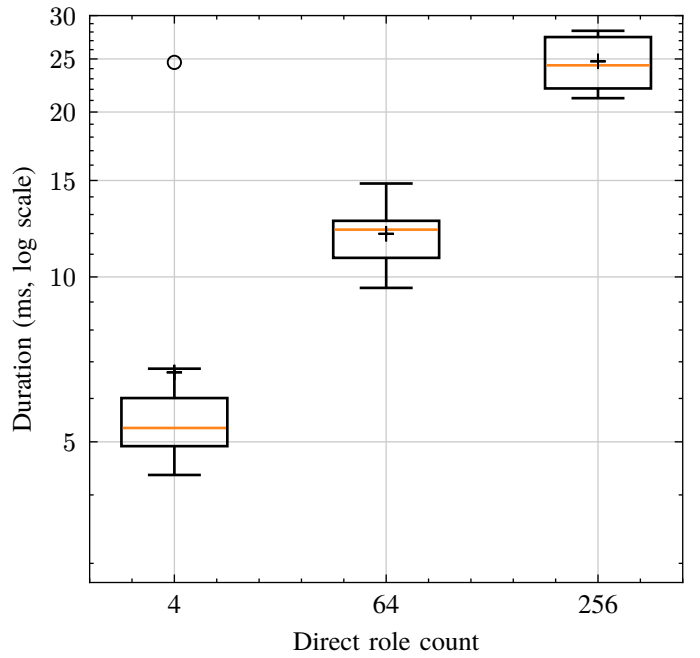


Fig. 6d. PermissionService box-plot distribution per parameter set (logarithmic y axis)

Figure 6. Box-plots for micro-benchmarks. Note that the y axes are not the same and some of the plots are logarithmic

S3-endpoint to an instance of OSWS. The test suite shows that all three tools are successfully able to query a Parquet file and perform operations on the result. The users are identified through their credentials, and columns that the users' roles do not give access to are masked correctly.

This demonstrates that OSWS works as a drop-in replacement for S3 when working with query engines that fetch Parquet files directly and do not store metadata.

The same e2e test was tried to be done for DuckLake, but due to how OSWS modifies the Parquet file, it was not supported.

B. Benchmarking

a) Micro-Benchmarks:

In Figure 6, the box-plots for the micro-benchmarks are shown, and it is important to note that not all y axes use the same scale, and some use logarithmic scaling.

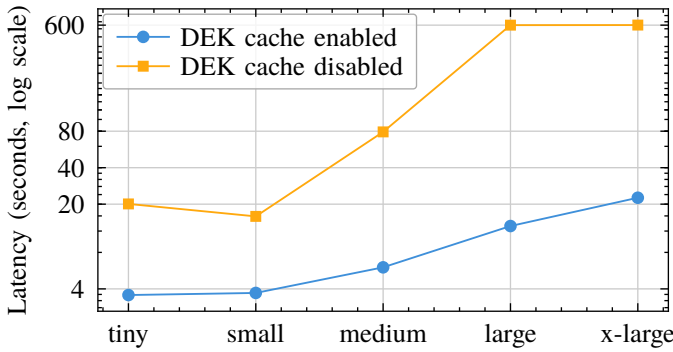


Figure 7. DEK cache impact on median cold GET latency. (logarithmic scale y axis)

The latency for Permission Hierarchy Benchmark, see Figure 6c, and Permission Service Benchmark, see Figure 6d, both have a minor contribution to latency, which is expected for RBAC-related calls.

The major contributors to latency are decryption, see Figure 6a, and DEK unwrapping, see Figure 6b.

Latency for *DEK unwrapping* does not grow much across the different key sizes. This is expected as the DEKs are unwrapped asynchronously, and the number of columns remains constant. Also, the time is not something that can be improved much besides changing the KV service provider to a faster alternative, and then, as OSWS currently does, cache the unwrapped DEK to use KV as little as possible. The values can be seen in more details in more details in Appendix 1.b, but essentially the *p95* time for unwrapping key size 256 of a tiny file is 4.11 seconds, with an average of 3.92 seconds, meaning that caching the unwrapped DEKs is essential, as if not cached this time would have to be added to each time a column has to be decrypted.

- Tiny had a *p95* latency of 17.1 milliseconds and an average of 13.6 milliseconds
- Medium had a *p95* latency of 1.8 seconds and an average of 1.68 seconds
- X-Large had a *p95* latency of 15.5 seconds and an average of 14.7 seconds

So, essentially, to make OSWS usable for larger files, fundamental design changes are needed in the decryption step.

The large numbers for the decryptions are due to a few design decisions. OSWS relies on Parquet Sharp, which, as mentioned, makes it necessary, regardless of permission level, to copy over the whole Parquet file. To optimize this, a custom low-level reader would be needed, which could decrypt the columns in place. This would help, as the current decryption micro-benchmark also takes the copying into account, though the main improvement would be seen when columns are not authorized to the client, and can thus be left as is. So instead of the computations being directly related to the number of Parquet columns, they will be related to the number of authorized columns.

b) E2E Benchmarks:

To begin with, a comparison of cold GET latency, with and without the DEK cache enabled, is shown in Figure 7. It can be seen that a ceiling is hit at ~600 seconds for the *large* and *x-large* Parquet files, without DEK cache enabled. This can

be attributed to the benchmarking timing out. This shows that on files at ~500MB and ~1GB, the system is unusably slow without the DEK cache. However, there is a huge difference even for smaller files. For the *tiny* GET, it was about ~5.6× slower, and for *medium*, it was about times ~13× slower.⁹

At first, it seemed surprising that there was such a large difference in having the DEK cache enabled, given that the cache is *cold* in this benchmark. However, this can be explained by the DEK cache warming up *as the file is being read*, since reading a row chunk will populate the DEK cache with the given columns of that chunk. Since the whole column is encrypted with the same DEK, once that column is reached again in a new row chunk, the cache is warm. These results show that the DEK cache is a must-have for OSWS. For that reason, the **no DEK cache** configuration is omitted from here on.

Looking at Figure 8a for a warm GET, it is again clear that the DEK cache decreases latency noticeably. On the *tiny* file size, all OSWS configurations sit at around ~30 milliseconds, thus being comparatively fast to larger file sizes. However, the relative overhead compared to direct fetch is still significant, with the direct fetch about ~10 milliseconds. Specifically, the results show an overhead of ~2.5× for OSWS with caching and direct fetch, with an average of 30 and 12 milliseconds, respectively.¹⁰ While the relative overhead is large, OSWS at 30 milliseconds remains practically useful.

Interestingly, the **no encrypt** is seemingly a bit slower than the other configurations for the *tiny* and *small* configuration, even though it should just be the pure network overhead. However, disabling encryption also disables the file cache, which might be why it is slower – due to it having to go to Digital Ocean every time. This is hinted at by it being very close to the **no file cache** configuration. But this is mostly speculation and would require further benchmarks to prove. More strangely, the *x-large* size on **warm GET** is seemingly much slower than **cold GET**. This result does not make much sense and should likely be disregarded entirely. This can be due to various factors, but is reasoned to be due to data fragmentation increasing over the benchmark runs. Lastly, reading from the data lake and the encrypted file cache might also have some unforeseen consequences, as they respectively use a memory stream and a file stream. But as the main latencies in the system were related to encryption/decryption and KV, this was down-prioritized.

In Figure 8b cold GET is shown. Direct fetch is omitted here due to there being no *cold* path – this is the same as Figure 8a.

Here, the **no encryption** configuration is much faster than the others for smaller files, at almost two orders of magnitude faster for the *tiny* file.

For OSWS with and without file caching, there is generally little difference in latency from the *tiny* to *medium* files. This highlights the overhead of fetching DEKs from the KV and performing decryption itself. However, as the files get larger, the network overhead of simply fetching the file starts to

⁹Calculated from the results shown in Appendix 1.c

¹⁰See Appendix 1.c for full results

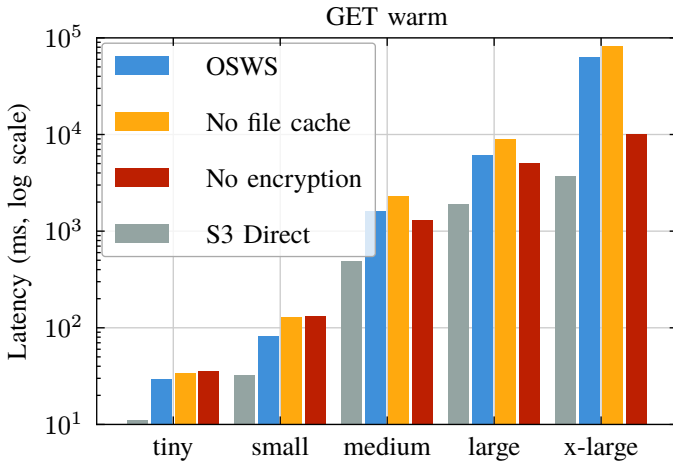


Fig. 8a. Median GET latency on warm cache

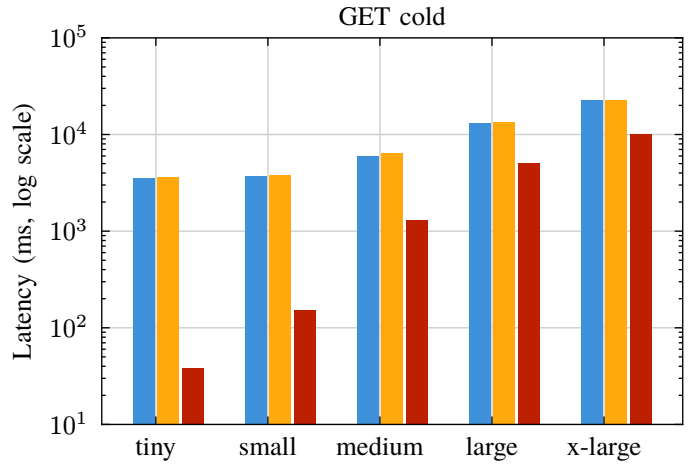


Fig. 8b. Median GET latency on cold cache

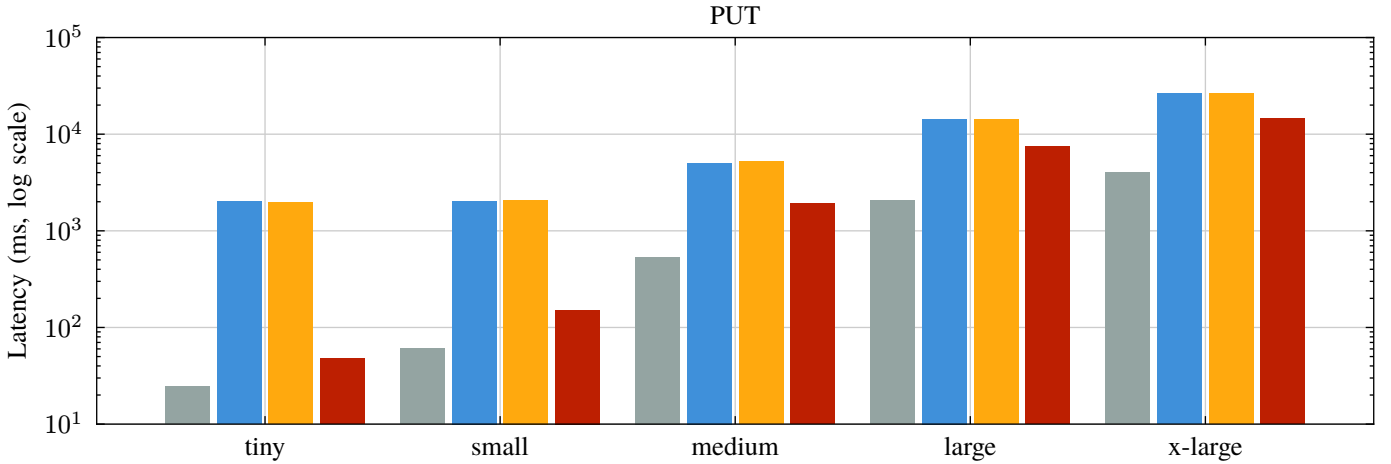


Fig. 8c. Median PUT latency

Figure 8. Plots showing median e2e latency measured during benchmarks. Grouped by file size. Note that legend is shared. (logarithmic y axis)

show as well, when the **no encryption** configuration starts to approach the other configurations.

Both Figure 8a and Figure 8b also show that the file cache is mostly useful when files are large; this is not surprising, as the encrypted file cache only helps with reducing S3-fetches, as the file must still be decrypted again. But when the network cost of fetching starts getting meaningful, the file cache matters.

Figure 8c shows the PUT latency. Unsurprisingly, there is massive overhead on the *tiny* configuration, with the direct upload at around ~30 milliseconds, and OSWS at ~3000 milliseconds. This is explained by the direct upload being pure network latency to the Object Store, while OSWS must spend time encrypting the file and making calls to KV, on top of uploading the resulting file. Again, as files get larger, the relative overhead reduces as the network cost of uploading becomes the dominating factor.

VII. DISCUSSION

This section will provide a discussion about some of the major issues behind the design choices made, and how they could be improved in a future redesign. In particular, the

choice of using PME along with Parquet Sharp introduced many issues, which will be addressed.

A. Limitations of the Current Design

The core issue of the current solution stems from the fact that the Parquet files put into OSWS are copied from the original Parquet file into a new one, instead of modifying the original.

When the Parquet file is copied, using Parquet Sharp, the resulting file will be inherently different from the original one, even without taking the encrypted data into account. Different implementations of Parquet writers cannot be guaranteed to produce the same file structure; metadata fields like `created_by` might be different or there might be different choices of encoding. This means when the writer ultimately fetches the file again, they cannot be sure it is the same file as they inserted, which introduces a problem for range requests.

The other issue with Parquet Sharp and the way OSWS stores the cryptographic metadata is that to decrypt the Parquet file, the full one first has to be fetched, even if the client only requests a specific range. When it has been fetched, the DEKs can be unwrapped, and now each column can be decrypted and copied over sequentially, and if there is

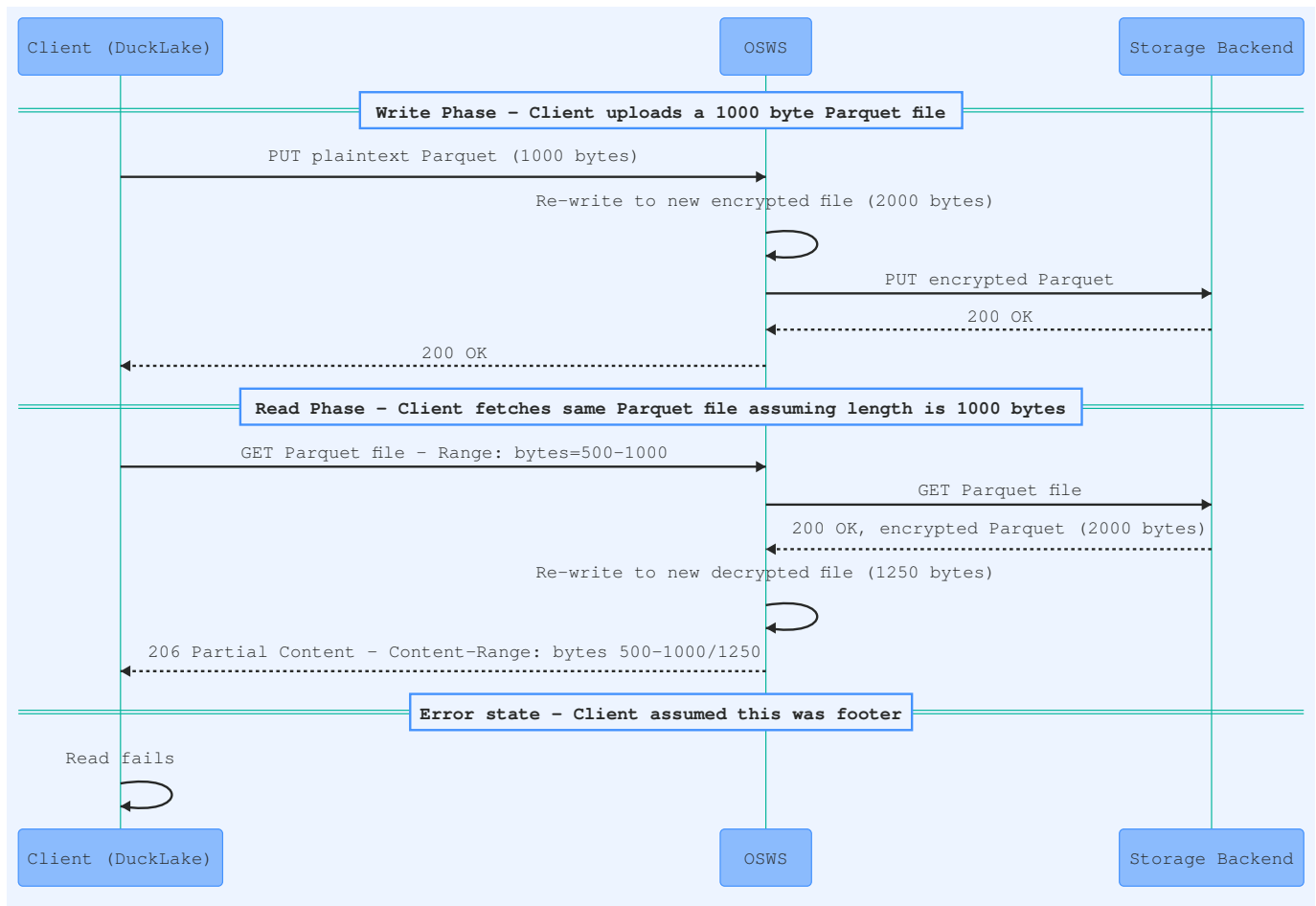


Figure 9. Sequence Diagram showing the DuckLake range request error case. File size and byte ranges are for illustrative purposes only.

no access, a dummy column has to be inserted in its place. Essentially, OSWS has to wait for each step to continue.

This results in OSWS not being able to return the original file, and the encrypted bytes for when a client is not authorized have to be replaced, but also that a lot of operations are not possible to overlap in its fetch, decrypt, and response pipeline.

B. Impact on Query Engines

The effect of the modified file size differs depending on whether a query engine is *schema-on-read* or *schema-on-write*.

Schema-on-read engines such as DuckDB and Spark are unaffected by the size change. When they fetch a file, they first issue a HEAD request to obtain the current file size, and then perform a range request for the footer using that size. Since this must report the size of the file that would be issued on a regular GET, a HEAD also triggers a full fetch and decrypt, including masking unauthorized columns. The size of this newly decrypted file is then reported to the query engine, ensuring that subsequent range requests are correct.

Schema-on-write tools that maintain a catalogue database, such as DuckLake [43], exhibit broken behaviour.

When DuckLake writes a Parquet file, it records both the file and footer size in its catalogue, allowing it to skip

the HEAD request on subsequent reads and fetch the footer directly.

For OSWS, however, the written file changes after the client PUT completes, so the recorded ranges are no longer valid. See Figure 9 for an illustration of this issue. The following describes it in more detail.

To begin, DuckLake uses its internal Parquet writer *duck-lake-writer* to create and PUT a Parquet file of 1000 bytes. Then, OSWS uses its writer, Parquet Sharp, to create an encrypted copy. This is now a whole new Parquet file with encrypted columns, new metadata and possibly different encoding due to the different writers - illustrated by a file size of 2000 bytes. This file is then stored in the storage backend.

When DuckLake later queries the file, it issues a range request of bytes 500-1000, this being the range it thinks contains the footer.

Since OSWS must fetch the entire file to be able to decrypt, the 2000 byte file is first fetched, and then decrypted. However, on decryption, the file is again copied into a whole new Parquet file. Now, the values are the same as the original, DuckLake-written 1000 byte file, but again, since the writers are not the same, the metadata and encoding might be different. This is illustrated with a file size of 1250 bytes.

To serve the range request, OSWS then slices *this* file in the range 500-1000, but given the different file size, this results in

a wrong range. This is then sent to DuckLake, which throws an error due to not finding the expected data.

This eliminates DuckLake and probably Snowflake from being compatible with OSWS in its current form. The root cause is not the specific metadata that is added, but the fundamental incompatibility between recording file ranges at write time and then rewriting the file.

C. Proposed Redesign

The issues described above share a common fix: OSWS should neither store cryptographic metadata inside the Parquet file, nor modify the size of the file at all. This requires two coordinated changes: an internal metadata store and a custom Parquet modifier using a length-preserving encryption scheme like AES-CTR.

a) Internal Metadata Store:

Cryptographic metadata should be stored in an internal SQL database within OSWS rather than in the Parquet file column metadata. For each Parquet file, the database should contain:

- 1) Internal file identifier for the footer¹¹
- 2) Byte range of the start and end index
- 3) Internal file identifier for the column index¹¹
- 4) Wrapped DEK
- 5) KEK reference
- 6) Row chunk size

When a read request arrives, OSWS can retrieve the file from the Object Store while concurrently querying the internal database for matching entries under the given identifier. It can then cross-reference the client's RBAC permissions, match authorized columns against the internal identifier and column index, and unwrap the DEKs with KV using the KEK reference. For range requests, OSWS no longer needs to fetch the full file, but can instead fetch the same range requested. Then, the internal metadata store can be used to determine what columns this range covers, thus knowing what DEKs will be needed to decrypt.

b) Custom Modifier with AES-CTR:

The current dependency on Parquet Sharp should be replaced with a custom Parquet modifier that encrypts and decrypts in place using AES-CTR. With this in place, there is also no need to rely on PME. AES-CTR uses no padding and, in contrast to AES-GCM, adds no authentication tag. This means the encrypted form of a column chunk is the same size as the plaintext, preserving all byte offsets and leaving the file size unchanged. Since there is no need to rewrite the file, compression can also be left intact.

This enables three concrete improvements over the current design:

- 1) *Asynchronous key pre-fetching*: While the encrypted Parquet file is being streamed from the Object Store, OSWS can concurrently request DEK unwrapping from KV. By the time the file has arrived, the unwrapped keys are ready, removing KV latency from the critical path.

- 2) *In-place decryption*: Rather than copying the entire file, only the requested column chunks are decrypted, using AES-CTR's preserved byte offsets to locate them directly.

Ultimately, Parquet Modular Encryption is not the right fit for OSWS, as it was built for an entirely different use case and trust model. PME was built for a scenario where the client handles all encryption and decryption themselves, as they do not trust the storage server. In this scenario, it makes sense to store cryptographic metadata in the Parquet file itself; when it is later read, the client knows how to decrypt it immediately.

For OSWS, the scenario is different. Here, the client trusts OSWS to handle cryptography and access control. Since OSWS is the sole handler of cryptography, it does not need to adhere to the specification set by PME, and can thus store metadata elsewhere and encrypt using its own strategy – as long as it correctly handles access control and the original writer can get the same file back when later fetched. Thus, PME should be discarded in favour of a fully custom modifier.

Additionally, knowing the column chunk-size and row-chunk size from the internal store allows OSWS to identify the precise offset within a range query, fetch only the relevant bytes from S3, and decrypt only that portion, which substantially reduces overhead compared to the current full-file fetch.

c) Parallelization:

With in-place encryption and decryption, row-group processing becomes an embarrassingly parallelizable operation. OSWS can spawn threads to process independent row groups concurrently rather than sequentially, which should reduce latency substantially. This was not feasible with Parquet Sharp, which provides no API for writing bytes at a specific offset.

D. Encrypted Parquet Storage Cache

When Parquet files are fetched from S3, they are stored in local storage in their encrypted format. As seen in Section VI.B.b, this yields negligible improvement for smaller files, with more significant gains on larger ones. With proper range request support, this improvement is likely to diminish further, since most requests will cover only a portion of a file rather than fetching it in full. Caching in decrypted form in-memory would offer additional performance gains, but it introduces complexity around column-level access control and memory pressure. This is therefore considered a later-stage optimization to be evaluated on a future OSWS system.

E. Correct Decisions & OSWS Future

Despite the design issues outlined above, several decisions made in the current OSWS implementation are considered sound and should be carried over to a V2 system.

a) Envelope Encryption:

Envelope encryption should be retained. While it carries a significant initial overhead, this is substantially mitigated by the decrypted DEK cache. A fully fledged key vault inside OSWS would be an alternative that eliminates the KV dependency, but would require significant additional security measures.

¹¹Defined within [44]

b) *DEK Cache*:

The decrypted DEK cache must be carried over to any future version that uses a KV store. As seen in both Section VI.B.b and Section VI.B.a, DEK unwrapping carries a significant overhead, and the cache is the primary mechanism for keeping it off the critical path on repeated accesses.

c) *RBAC Metadata Storage*:

The micro-benchmarks in Section VI.B.a showed that authorizing users via the RBAC database contributes negligible overhead relative to cryptographic operations and network transfer. A PostgreSQL database with the implemented schema is therefore a sound choice for RBAC metadata storage going forward.

d) *Role Management*:

Role management – creating and assigning roles, granting permissions – using the implemented query editor provided an intuitive and familiar interface over the admin API. While this is a wrapper that could be implemented in many ways, it is a valid option for future versions.

e) *Known Security Considerations*:

Two implementation-level security issues are acknowledged in the current OSWS release, documented in `KNOWN_ISSUES.md`.¹²

First, S3 credential secret keys are stored in plaintext in the PostgreSQL database, as the SigV4 authentication handler requires the raw key for HMAC derivation. If the database is compromised, all S3 credentials become immediately usable.

This could be mitigated by encrypting secret keys at rest using Azure Key Vault and decrypting on read during SigV4 authentication.

Second, the RBAC admin flag is provisioned just-in-time from OIDC claims on every login. If the OIDC provider exposes this as a self-service field, users could grant themselves admin access. This is dependent on the OIDC provider configuration and can be mitigated by using a dedicated admin identity provider or restricting claim sources.

VIII. CONCLUSION

The proof-of-concept version OSWS demonstrates the feasibility of adding fine-grained role-based access control in data lakes by encryption with native Parquet files, and thus enforcing it within the Object Store. Query engines that do not cache Parquet sizing metadata, such as DuckDB, can use OSWS without modifications by pointing their S3 URL to OSWS.

This thesis had the following six objectives:

- 1) Create a PoC of OSWS using an underlying S3-compatible Object Store
- 2) Ensure OSWS enforces access control using RBAC and encryption
- 3) Make it compatible with “fully managed all-in-one cloud platforms”
- 4) Make it compatible with “external query engines”
- 5) Benchmark OSWS to measure the latency it adds.

- 6) Reflect on design decisions and which alternatives would have been better

Four of the six objectives were partially completed.

The OSWS PoC was successfully created and supports column-level RBAC using encryption.

Making it compatible with “fully managed all-in-one cloud platforms” was not possible to test, due to whitelisting limitations.

External query engines were able to be connected to the system without needing any modifications – though only for the ones that do not cache metadata on write. Meaning that objective 4 was only partially completed. DuckDB could use OSWS as a drop-in replacement for S3 without needing any modifications, but DuckLake was not able to use OSWS.

OSWS was benchmarked, and showed that a DEK cache is a non-negotiable, even with small Parquet files; going from no DEK cache entirely to a cold cache resulted in a $\sim 5.6\times$ improvement from 20.1 seconds on average to 3.6 seconds on average. Going from a cold to a warm DEK cache further improved latency by about $\sim 120\times$ down to 30 milliseconds. While still a $\sim 2.5\times$ overhead, it remains practically usable.

However, generally the benchmarks showed that OSWS in its current state is not production-ready, due to design choices; 1GB Parquet files made the system time out and for 125MB Parquet files, retrieving them in their best case would take 1.6 seconds.

The results of the current state of OSWS are better seen as a design proof that the suggested system is possible. A lot of design decisions have to be carried over into a potential future version of it, including: DEK caching, envelope encryption, KV provider agnostic, and OIDC provider agnostic. While the design decisions of using PME, Parquet Sharp, and the way metadata is handled have to be changed, to ensure a system supporting more asynchronous work and internally supported range requests. These ideas have been reflected upon, and a more detailed solution is proposed within Section VII.

REFERENCES

- [1] A. S. H. Trøstrup and L. F. T. Hanson, “Reviewing options for fine-grained Role-Based Access Control in Data Lakes.” [Online]. Available: <https://github.com/lucasfth/reviewing-options-for-fine-grained-role-based-access-control-in-data-lakes>
- [2] B. Dageville *et al.*, “The Snowflake Elastic Data Warehouse,” in *Proceedings of the 2016 International Conference on Management of Data*, San Francisco California USA: ACM, 2016, pp. 215–226. doi: 10.1145/2882903.2903741.
- [3] [Online]. Available: <https://www.databricks.com/product/unity-catalog>
- [4] S. Kumar, S. Yagati, C. Power, D. E. Culler, and R. A. Popa, “Membrane: A Cryptographic Access Control System for Data Lakes,” *ArXiv*, 2025, [Online]. Available: <https://api.semanticscholar.org/CorpusID:281243750>
- [5] X. Shang *et al.*, “One Stone, Three Birds: Finer-Grained Encryption with Apache Parquet @ Large Scale,” in *2022 IEEE International Conference on Big Data (Big Data)*, Osaka, Japan: IEEE, Dec. 2022, pp. 5802–5811. doi: 10.1109/BigData55660.2022.10020987.
- [6] A. Jonker and M. M. Kosinski, “What is a data lake?.” [Online]. Available: <https://www.ibm.com/think/topics/data-lake>
- [7] [Online]. Available: <https://www.snowflake.com/content/snowflake-site/global/en/fundamentals/understanding-structured-semi-structured-and-unstructured-data>
- [8] AWS, [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html>

¹²https://github.com/lucasfth/osws/blob/V2026.0.0-alpha/KNOWN_ISSUES.md

- [9] A. S. Foundation, "Apache Spark." [Online]. Available: <https://spark.apache.org/>
- [10] M. A. Zaharia, A. Ghodsi, R. Xin, and M. Armbrust, "Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics," in *Conference on Innovative Data Systems Research*, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:229576171>
- [11] Apache, "Parquet File Format." [Online]. Available: <https://parquet.apache.org/docs/file-format/>
- [12] A. S. Foundation, "Apache Thrift." [Online]. Available: <https://thrift.apache.org/>
- [13] Apache, [Online]. Available: <https://parquet.apache.org/docs/file-format/data-pages/encryption/>
- [14] D. F. Ferraiolo and R. Kuhn, "Role-Based Access Controls," in *Proceedings of the 15th National Computer Security Conference (NCSC)*, Baltimore, MD, USA, Oct. 1992, pp. 554–563.
- [15] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Trans. Inf. Syst. Secur.*, vol. 4, no. 3, pp. 224–274, Aug. 2001, doi: [10.1145/501978.501980](https://doi.org/10.1145/501978.501980).
- [16] O. Foundation, "What is OpenID Connect." [Online]. Available: <https://openid.net/developers/how-connect-works/>
- [17] Microsoft, "What is Microsoft Entra?." [Online]. Available: <https://learn.microsoft.com/da-dk/entra/fundamentals/what-is-entra>
- [18] PocketID, "Introduction to PocketID." [Online]. Available: <https://pocket-id.org/docs/introduction>
- [19] AWS, "AWS Key Management Service." [Online]. Available: <https://docs.aws.amazon.com/kms/latest/developerguide/overview.html>
- [20] Azure, "Azure Key Vault Overview - Azure Key Vault." [Online]. Available: <https://docs.azure.cn/en-us/key-vault/general/overview>
- [21] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," technical report SP800–38D, 2007. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/38/d/final>
- [22] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," technical report SP800–38A, 2001. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/38/a/final>
- [23] Google, [Online]. Available: <https://docs.cloud.google.com/kms/docs/envelope-encryption>
- [24] Redis, "TTL." [Online]. Available: <https://redis.io/docs/latest/commands/ttl/>
- [25] E. Barker, *Recommendation for key management: part 1 - general*, no. NIST SP 800-57pt1r5. Gaithersburg, MD, 2020, p. NIST SP 800–57pt1r5. doi: [10.6028/NIST.SP.800-57pt1r5](https://doi.org/10.6028/NIST.SP.800-57pt1r5).
- [26] [Online]. Available: https://duckdb.org/docs/current/core_extensions/httpfs/s3api
- [27] G. User, "Introduction." [Online]. Available: <https://ducklake.select/docs/stable/specification/introduction.html>
- [28] "Authenticating Requests (AWS Signature Version 4)." [Online]. Available: <https://docs.aws.amazon.com/AmazonS3/latest/API/sig-v4-authenticating-requests.html>
- [29] Deepstrike, [Online]. Available: <https://deepstrike.io/blog/what-is-jit-provisioning>
- [30] R. Anderson and T. Dykstra, "Tutorial: Create a Minimal API with ASP.NET Core." [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/tutorials/min-web-api?view=aspnetcore-10.0>
- [31] msmbaldwin, "Azure data encryption at rest - Azure Security." [Online]. Available: <https://learn.microsoft.com/en-us/azure/security/fundamentals/encryption-atrest>
- [32] [Online]. Available: <https://github.com/G-Research/ParquetSharp/releases>
- [33] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "PKCS #1: RSA Cryptography Specifications Version 2.2," technical report RFC8017, 2016. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc8017>
- [34] peggyjs, "peggyjs/peggy – Parser Generator for JavaScript." [Online]. Available: <https://github.com/peggyjs/peggy>
- [35] [Online]. Available: <https://github.com/G-Research/ParquetSharp>
- [36] "Reading and writing Parquet files." [Online]. Available: <https://arrow.apache.org/docs/cpp/parquet.html>
- [37] Snowflake, "Introduction to external tables." [Online]. Available: [https://docs.snowflake.com/en/user-guide/tables-external-intro?utm_source=](https://docs.snowflake.com/en/user-guide/tables-external-intro?utm_source=DV-Blog%3Fsource&utm_cta=website-workload-ai-ml-timely-content-building-a-chatbot-app-with-rag-and-feature-store)
- [38] Snowflake, "External table files." [Online]. Available: https://docs.snowflake.com/en/sql-reference/functions/external_table_files
- [39] ClickHouse, "ClickHouse Example Datasets." [Online]. Available: <https://clickhouse.com/docs/getting-started/example-datasets>
- [40] T. W. Schneider, "Analyzing 1.1 Billion NYC Taxi and Uber Trips, with a Vengeance." [Online]. Available: <https://toddwtschneider.com/posts/analyzing-1-1-billion-nyc-taxi-and-uber-trips-with-a-vengeance/>
- [41] "Companies using MinIO in 2026 | Landbase." Accessed: Mar. 23, 2026. [Online]. Available: <https://data.landbase.com/technology/minio/>
- [42] "Companies that use Minio (2,450)." Accessed: Mar. 23, 2026. [Online]. Available: <https://theirstack.com/en/technology/minio>
- [43] DuckDB Foundation, "ducklake_data_file." [Online]. Available: https://ducklake.select/docs/stable/specification/tables/ducklake_data_file.html
- [44] "Parquet Modular Encryption." [Online]. Available: <https://parquet.apache.org/docs/file-format/data-pages/encryption/>

I. APPENDIX

A. VM Specification

```
root@ubuntu-s-1vcpu-1gb-35gb-intel-fral:~# inxi -Fxz
System:
  Kernel: 6.8.0-111-generic arch: x86_64 bits: 64 compiler: gcc v: 13.3.0
  Console: pty pts/6 Distro: Ubuntu 24.04.4 LTS (Noble Numbat)
Machine:
  Type: Kvm Mobo: DigitalOcean model: Droplet v: 20171212 serial: <filter> BIOS:
DigitalOcean
  v: 20171212 date: 12/12/2017
CPU:
  Info: 8-core model: DO-Premium-AMD bits: 64 type: MCP arch: Zen 2 rev: 0 cache: L1:
512 KiB
  L2: 4 MiB L3: 16 MiB
  Speed (MHz): avg: 1996 min/max: N/A cores: 1: 1996 2: 1996 3: 1996 4: 1996 5: 1996
6: 1996
  7: 1996 8: 1996 bogomips: 31940
  Flags: avx avx2 ht lm nx pae sse sse2 sse3 sse4_1 sse4_2 sse4a ssse3 svm
Graphics:
  Device-1: Red Hat Virtio 1.0 GPU driver: virtio-pci v: 1 bus-ID: 00:02.0
  Display: server: No display server data found. Headless machine? tty: 110x29
  resolution: 1024x768
  API: EGL v: 1.5 drivers: kms_swrast,swrast platforms: active:
gbm,surfaceless,device
  inactive: wayland,x11
  API: OpenGL v: 4.5 vendor: mesa v: 25.2.8-0ubuntu0.24.04.1 note: console (EGL
sourced)
  renderer: llvmpipe (LLVM 20.1.2 256 bits)
Audio:
  Message: No device data found.
  API: ALSA v: k6.8.0-111-generic status: inactive
Network:
  Device-1: Intel 82371AB/EB/MB PIIX4 ACPI vendor: Red Hat Qemu virtual machine
  type: network bridge driver: N/A port: N/A bus-ID: 00:01.3
  Device-2: Red Hat Virtio network driver: virtio-pci v: 1 port: clc0 bus-ID: 00:03.0
  IF: eth0 state: up speed: -1 duplex: unknown mac: <filter>
  Device-3: Red Hat Virtio network driver: virtio-pci v: 1 port: clc0 bus-ID: 00:04.0
  IF: eth1 state: up speed: -1 duplex: unknown mac: <filter>
  IF-ID-1: br-cf3f443f079d state: down mac: <filter>
  IF-ID-2: docker0 state: down mac: <filter>
Drives:
  Local Storage: total: 320 GiB used: 19.4 GiB (6.1%)
  ID-1: /dev/vda model: N/A size: 320 GiB
  ID-2: /dev/vdb model: N/A size: 488 KiB
Partition:
  ID-1: / size: 308.93 GiB used: 19.28 GiB (6.2%) fs: ext4 dev: /dev/vda1
  ID-2: /boot size: 880.4 MiB used: 113.9 MiB (12.9%) fs: ext4 dev: /dev/vda16
  ID-3: /boot/efi size: 104.3 MiB used: 6.1 MiB (5.9%) fs: vfat dev: /dev/vda15
Swap:
  Alert: No swap data was found.
Sensors:
  Src: lm-sensors+/sys Message: No sensor data found using /sys/class/hwmon or lm-
sensors.
Info:
  Memory: total: 16 GiB available: 15.62 GiB used: 1.03 GiB (6.6%)
  Processes: 180 Uptime: 4d 1h 2m Init: systemd target: graphical (5)
  Packages: 813 Compilers: N/A Shell: Bash v: 5.2.21 inxi: 3.3.34
```

Listing 3. VM specification used for the benchmarks of OSWS

B. Micro Benchmark Results

Benchmark: Decryption								
parameters	n	mean	stddev	min	p50	p95	p99	max
size=large	15	7.48s	313.4ms	7.12s	7.43s	8.17s	8.17s	8.17s
size=medium	15	1.68s	64.1ms	1.58s	1.67s	1.80s	1.80s	1.80s
size=small	15	65.7ms	4.8ms	58.7ms	64.2ms	75.0ms	75.0ms	75.0ms
size=tiny	15	13.6ms	1.2ms	12.0ms	13.4ms	17.1ms	17.1ms	17.1ms
size=xlarge	15	14.7s	452.1ms	14.1s	14.6s	15.5s	15.5s	15.5s
Benchmark: KeyUnwrap								
parameters	n	mean	stddev	min	p50	p95	p99	max
size=small,dek_bits=128	15	4.15s	166.5ms	3.94s	4.15s	4.55s	4.55s	4.55s
size=small,dek_bits=192	15	4.05s	198.0ms	3.76s	3.99s	4.37s	4.37s	4.37s
size=small,dek_bits=256	15	4.05s	186.2ms	3.73s	4.04s	4.53s	4.53s	4.53s
size=tiny,dek_bits=128	15	3.15s	152.0ms	2.86s	3.12s	3.44s	3.44s	3.44s
size=tiny,dek_bits=192	15	3.24s	214.5ms	2.96s	3.22s	3.76s	3.76s	3.76s
size=tiny,dek_bits=256	15	3.92s	113.8ms	3.67s	3.97s	4.11s	4.11s	4.11s
Benchmark: PermissionHierarchy								
parameters	n	mean	stddev	min	p50	p95	p99	max
hierarchy_depth=0	15	4.3ms	0.7ms	3.5ms	4.2ms	5.8ms	5.8ms	5.8ms
hierarchy_depth=16	15	6.8ms	2.3ms	5.1ms	6.2ms	14.8ms	14.8ms	14.8ms
hierarchy_depth=4	15	4.5ms	0.8ms	3.6ms	4.4ms	7.0ms	7.0ms	7.0ms
hierarchy_depth=64	15	18.5ms	2.0ms	14.7ms	18.8ms	23.6ms	23.6ms	23.6ms
Benchmark: PermissionService								
parameters	n	mean	stddev	min	p50	p95	p99	max
role_count=256	15	24.7ms	2.6ms	21.2ms	24.3ms	28.1ms	28.1ms	28.1ms
role_count=4	15	6.7ms	4.8ms	4.3ms	5.3ms	24.6ms	24.6ms	24.6ms
role_count=64	15	12.0ms	1.4ms	9.6ms	12.2ms	14.8ms	14.8ms	14.8ms

Listing 4. Textual representation for the results of running the micro benchmarks

C. E2E Latency Benchmark Results

PUT cache=n/a						
config	metric	tiny	small	medium	large	xlarge
s3-direct	mean	27ms	69ms	575ms	2031ms	4093ms
	stddev	11ms	24ms	112ms	39ms	114ms
	p50	24ms	60ms	535ms	2046ms	4049ms
	p95	57ms	140ms	846ms	2076ms	4317ms
	n	10	10	10	10	10
osws-encrypt-cache	mean	2208ms	2024ms	5024ms	14.2s	26.5s
	stddev	660ms	94ms	161ms	354ms	466ms
	p50	1994ms	2034ms	5046ms	14.3s	26.5s
	p95	4168ms	2236ms	5234ms	14.7s	27.2s
	n	10	10	10	10	10
osws-encrypt-no-file-cache	mean	2150ms	2090ms	5228ms	14.4s	26.1s
	stddev	584ms	81ms	139ms	353ms	557ms
	p50	1965ms	2081ms	5236ms	14.2s	26.2s
	p95	3884ms	2222ms	5495ms	15.2s	27.2s
	n	10	10	10	10	10
osws-encrypt-no-dek-cache	mean	2558ms	2327ms	5152ms	14.2s	25.3s
	stddev	567ms	264ms	207ms	197ms	772ms
	p50	2383ms	2263ms	5054ms	14.2s	25.4s
	p95	4246ms	2845ms	5554ms	14.6s	26.7s
	n	10	10	10	10	10
osws-no-encrypt	mean	172ms	153ms	1940ms	7456ms	14.4s
	stddev	339ms	9ms	104ms	291ms	244ms
	p50	48ms	151ms	1928ms	7476ms	14.4s
	p95	1186ms	172ms	2165ms	7868ms	14.9s
	n	10	10	10	10	10

Listing 5. Textual representation of E2E Latency Benchmark results for PUT

GET cache=cold

config	metric	tiny	small	medium	large	xlarge
osws-encrypt-cache	mean	3592ms	3708ms	6018ms	13.1s	29.4s
	stddev	193ms	94ms	131ms	391ms	11.0s
	p50	3558ms	3703ms	6024ms	13.2s	22.6s
	p95	3997ms	3893ms	6239ms	13.8s	46.9s
	n	10	10	10	10	10
osws-encrypt-no-file-cache	mean	3662ms	3819ms	6436ms	13.6s	29.8s
	stddev	116ms	160ms	283ms	248ms	11.3s
	p50	3648ms	3819ms	6362ms	13.6s	22.7s
	p95	3863ms	4166ms	7007ms	14.1s	47.2s
	n	10	10	10	10	10
osws-encrypt-no-dek-cache	mean	20.1s	16.2s	79.6s	600.6s	600.7s
	stddev	797ms	811ms	3156ms	369ms	387ms
	p50	20.1s	15.9s	79.0s	600.5s	600.9s
	p95	21.8s	18.3s	85.6s	601.2s	601.1s
	n	10	10	10	10	10
osws-no-encrypt	mean	45ms	151ms	1303ms	5017ms	10.5s
	stddev	17ms	17ms	50ms	107ms	491ms
	p50	38ms	153ms	1300ms	5016ms	10.2s
	p95	92ms	177ms	1390ms	5220ms	11.6s
	n	10	10	10	10	10

GET cache=warm

config	metric	tiny	small	medium	large	xlarge
s3-direct	mean	12ms	33ms	477ms	1884ms	3715ms
	stddev	2ms	3ms	44ms	104ms	162ms
	p50	11ms	33ms	490ms	1915ms	3737ms
	p95	16ms	41ms	555ms	2015ms	3987ms
	n	10	10	10	10	10
osws-encrypt-cache	mean	30ms	82ms	1600ms	6040ms	65.6s
	stddev	3ms	6ms	101ms	248ms	5962ms
	p50	29ms	82ms	1617ms	6097ms	62.9s
	p95	37ms	92ms	1769ms	6362ms	75.0s
	n	10	10	10	10	10
osws-encrypt-no-file-cache	mean	35ms	130ms	2298ms	9090ms	82.2s
	stddev	3ms	9ms	57ms	274ms	2451ms
	p50	34ms	129ms	2282ms	9025ms	81.9s
	p95	42ms	151ms	2419ms	9610ms	87.8s
	n	10	10	10	10	10
osws-encrypt-no-dek-cache	mean	16.0s	15.4s	78.1s	508.5s	600.4s
	stddev	2087ms	531ms	4794ms	140.2s	291ms
	p50	14.6s	15.4s	76.5s	600.3s	600.4s
	p95	20.3s	16.3s	91.6s	600.8s	601.0s
	n	10	10	10	10	10
osws-no-encrypt	mean	38ms	138ms	1303ms	5214ms	9953ms
	stddev	5ms	16ms	43ms	297ms	434ms
	p50	36ms	133ms	1302ms	5103ms	10.0s
	p95	51ms	167ms	1378ms	5742ms	10.6s
	n	10	10	10	10	10

Listing 6. Textual representation of E2E Latency Benchmark results for GET with warm and cold cache